

기계학습

"기계가 일일이 코드로 명시하지 않은 동작을 데이터로부터 학습하여 실행할 수 있도록 하는 알고리즘을 개발하는 연구 분야 " **Arthur Samuel**[1959]

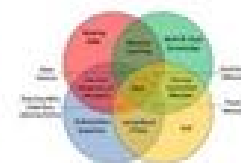
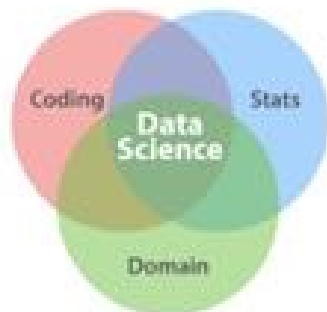
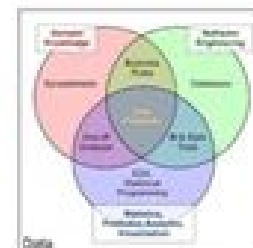
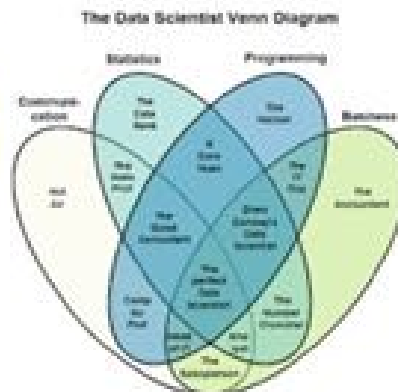
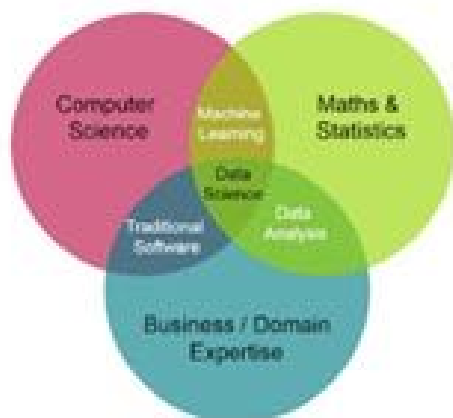
“Programming computers to learn from experience should eventually eliminate the need for much of this detailed programming effort. 컴퓨터가 경험을 통해 학습할 수 있도록 프로그래밍할 수 있다면, 세세하게 프로그래밍해야 하는 번거로움에서 벗어날 수 있다[Samuel1959].”

“A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

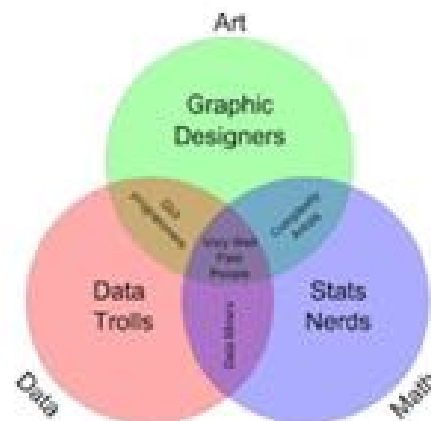
어떤 컴퓨터 프로그램이 T 라는 작업을 수행한다. 이 프로그램의 성능을 P 라는 척도로 평가했을 때 경험 E 를 통해 성능이 개선된다면 이 프로그램은 학습을 한다고 말할 수 있다[Mitchell1997(2쪽)].”

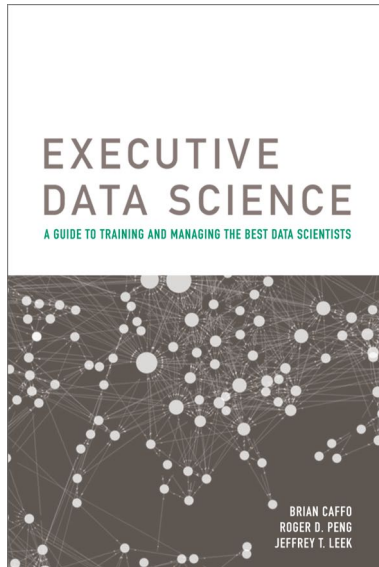
“Programming computers to optimize a performance criterion using example data or past experience 사례 데이터, 즉 과거 경험을 이용하여 성능 기준을 최적화하도록 프로그래밍하는 작업[Alpaydin2010]”

“Computational methods using experience to improve performance or to make accurate predictions 성능을 개선하거나 정확하게 예측하기 위해 경험을 이용하는 계산학 방법들[Mohri2012]”



Data Science





Occam's Razor



Machine learning

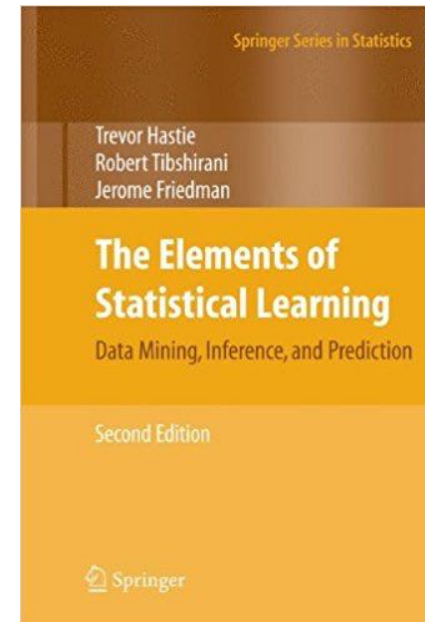
- build an automated movie recommender system
- success - anything that produces reliable recommendations

Statistical analysis

- build a parsimonious and interpretable model to better understand why people choose the movies that they do
- success - anything true learned about movie choices

Glossary

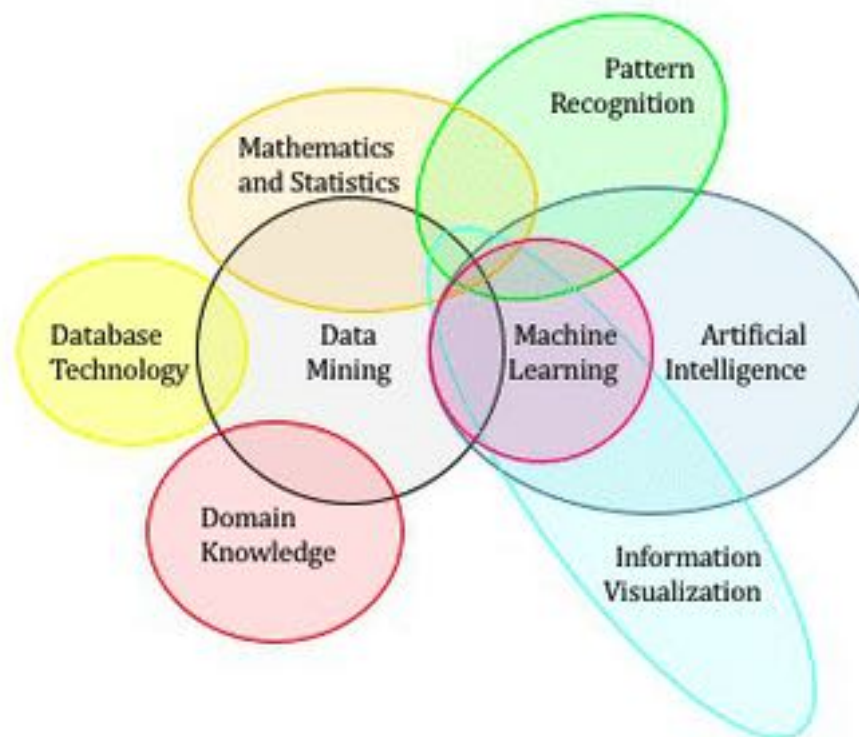
Machine learning	Statistics
network, graphs	model
weights	parameters
learning	fitting
generalization	test set performance
supervised learning	regression/classification
unsupervised learning	density estimation, clustering
large grant = \$1,000,000	large grant= \$50,000
nice place to have a meeting: Snowbird, Utah, French Alps	nice place to have a meeting: Las Vegas in August

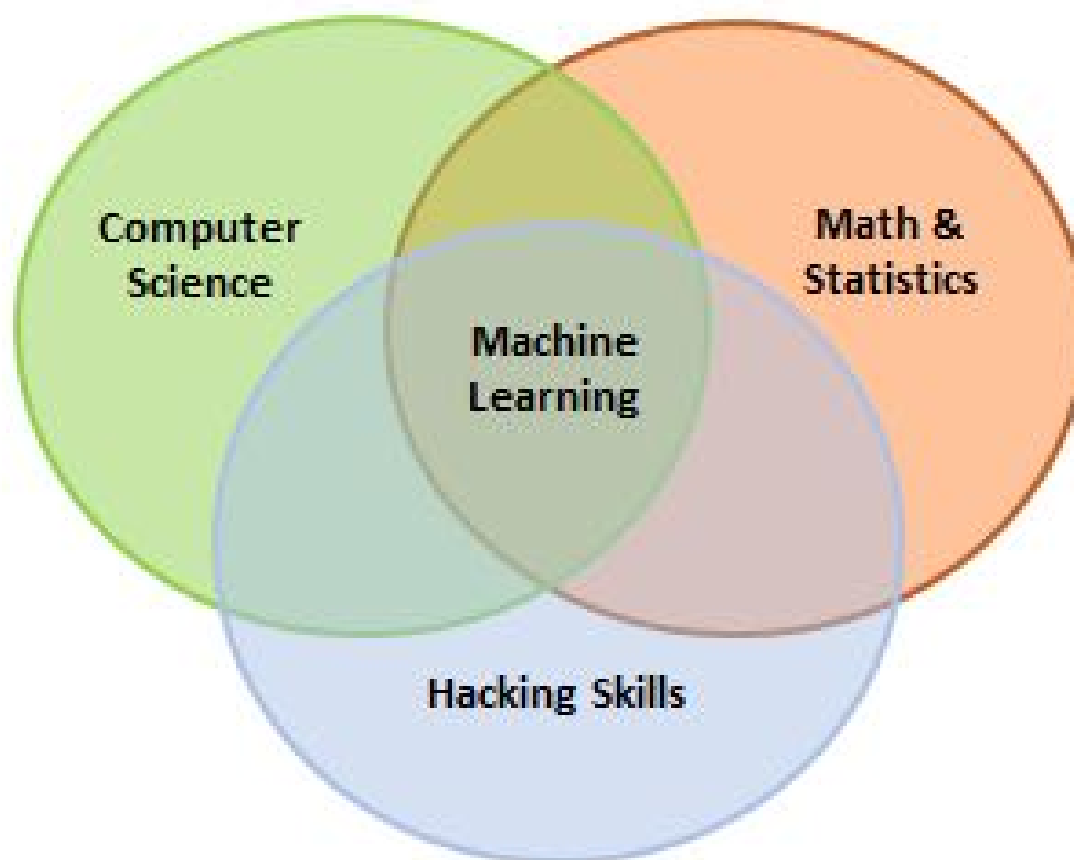


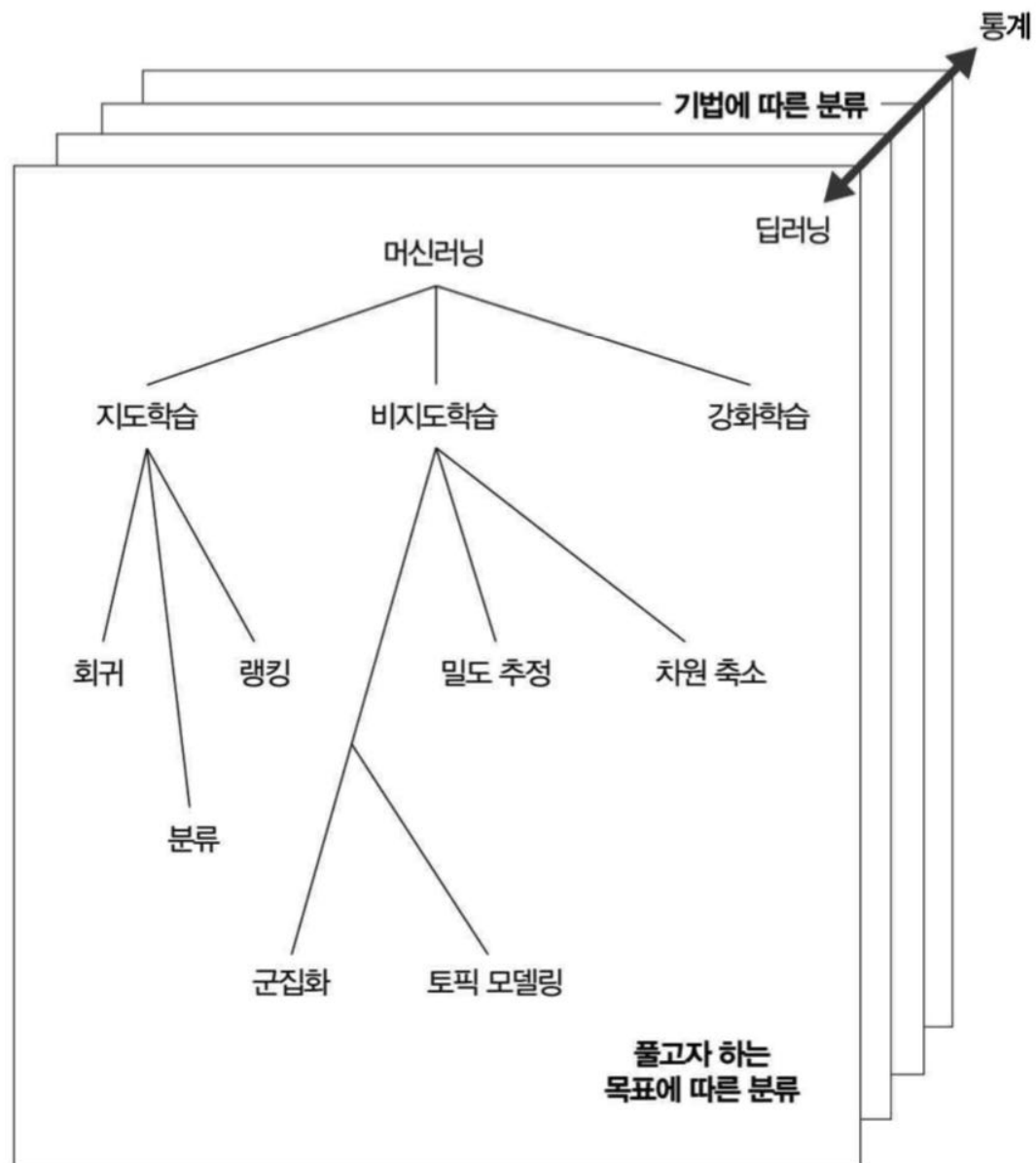
Machine Learning (AI/CS 분야, 딥러닝 등 Neural Network 기반)

VS

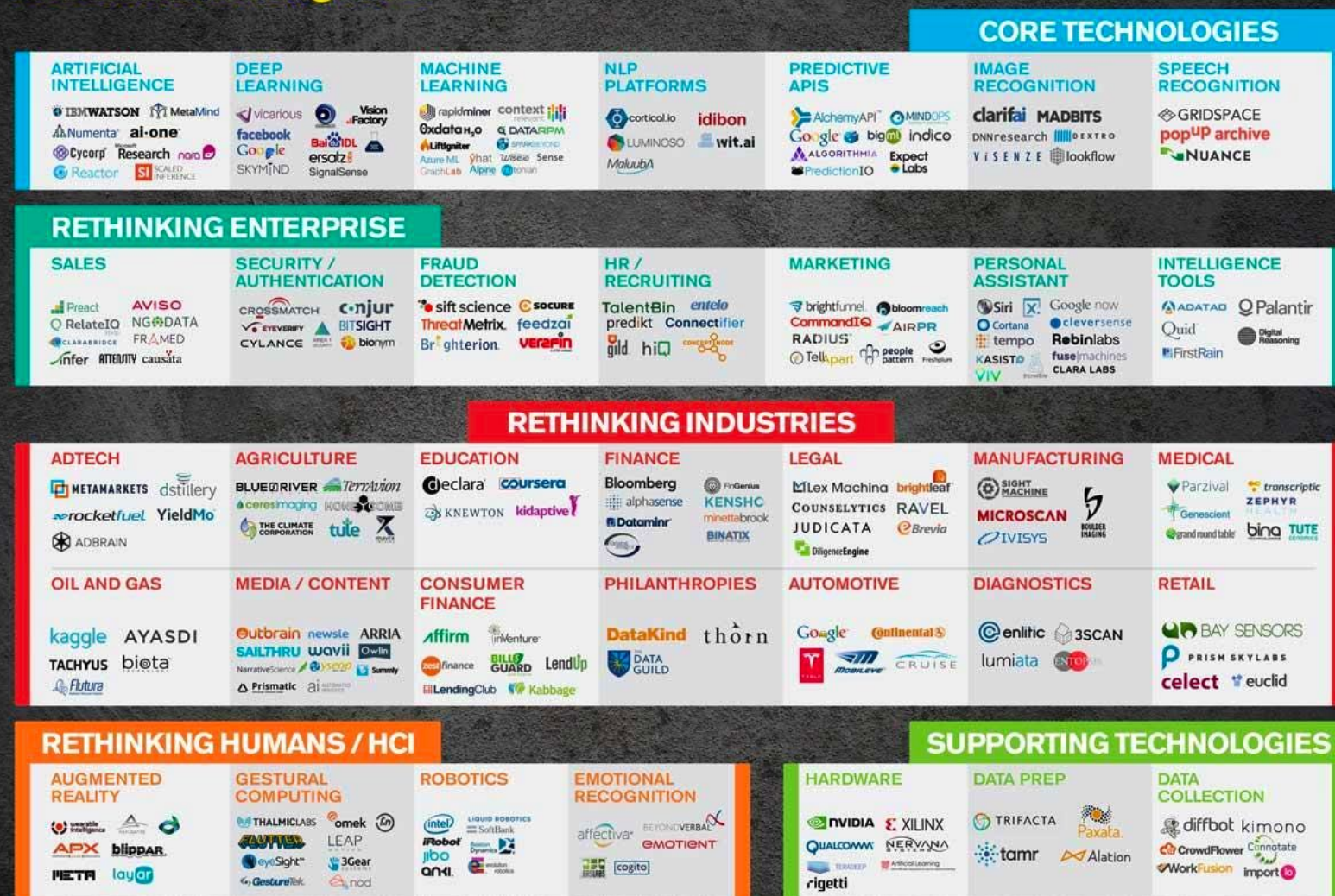
Statistical Analysis (Statistical Learning / Predictive Analysis)



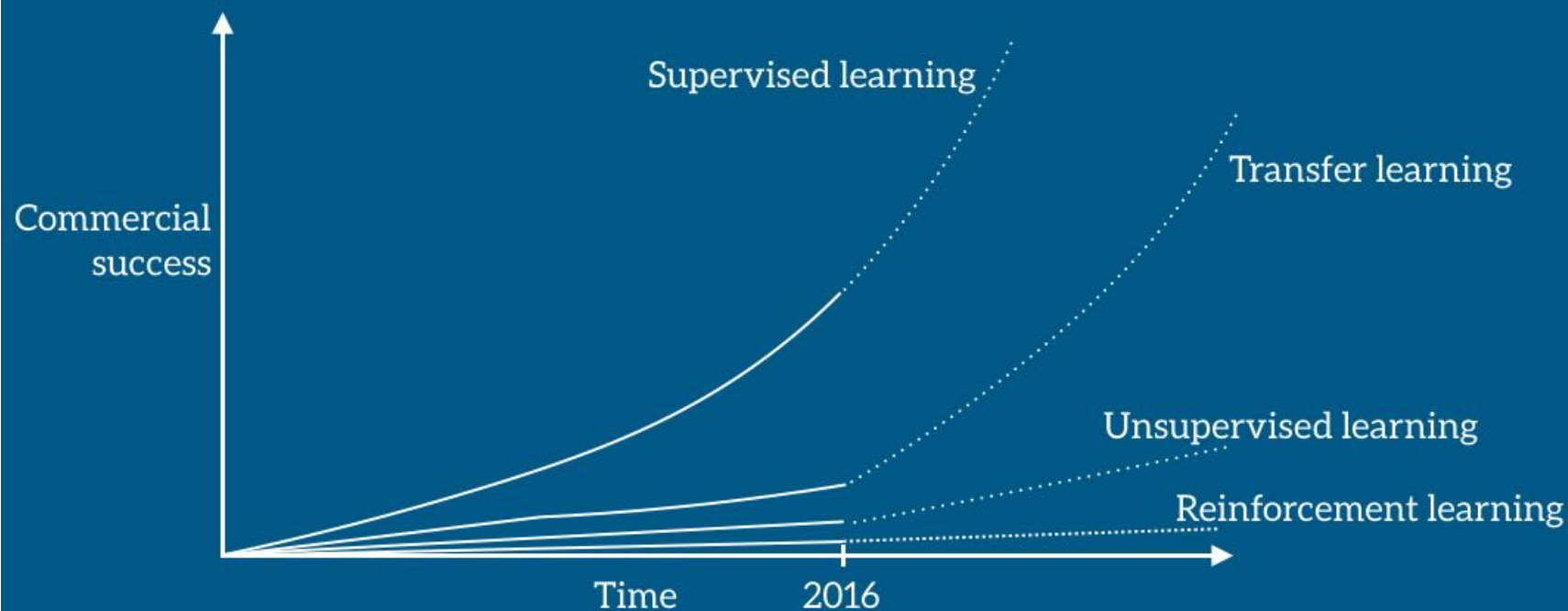




Machine Intelligence LANDSCAPE



Drivers of ML success in industry

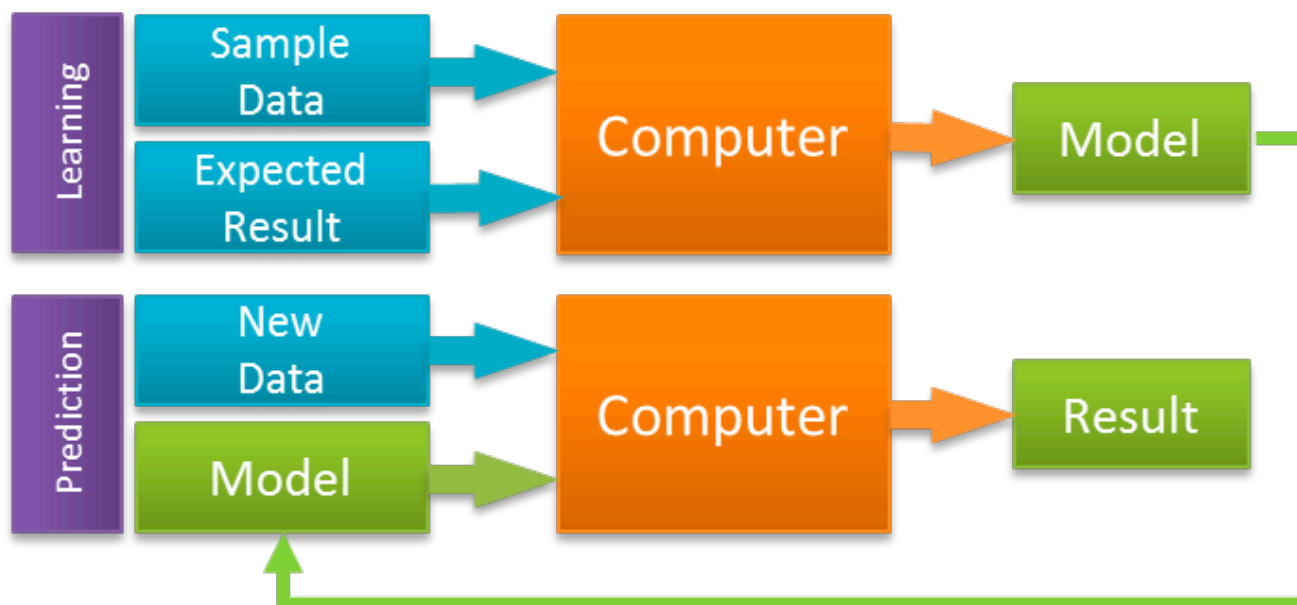


- Andrew Ng, NIPS 2016 tutorial

Traditional modeling:

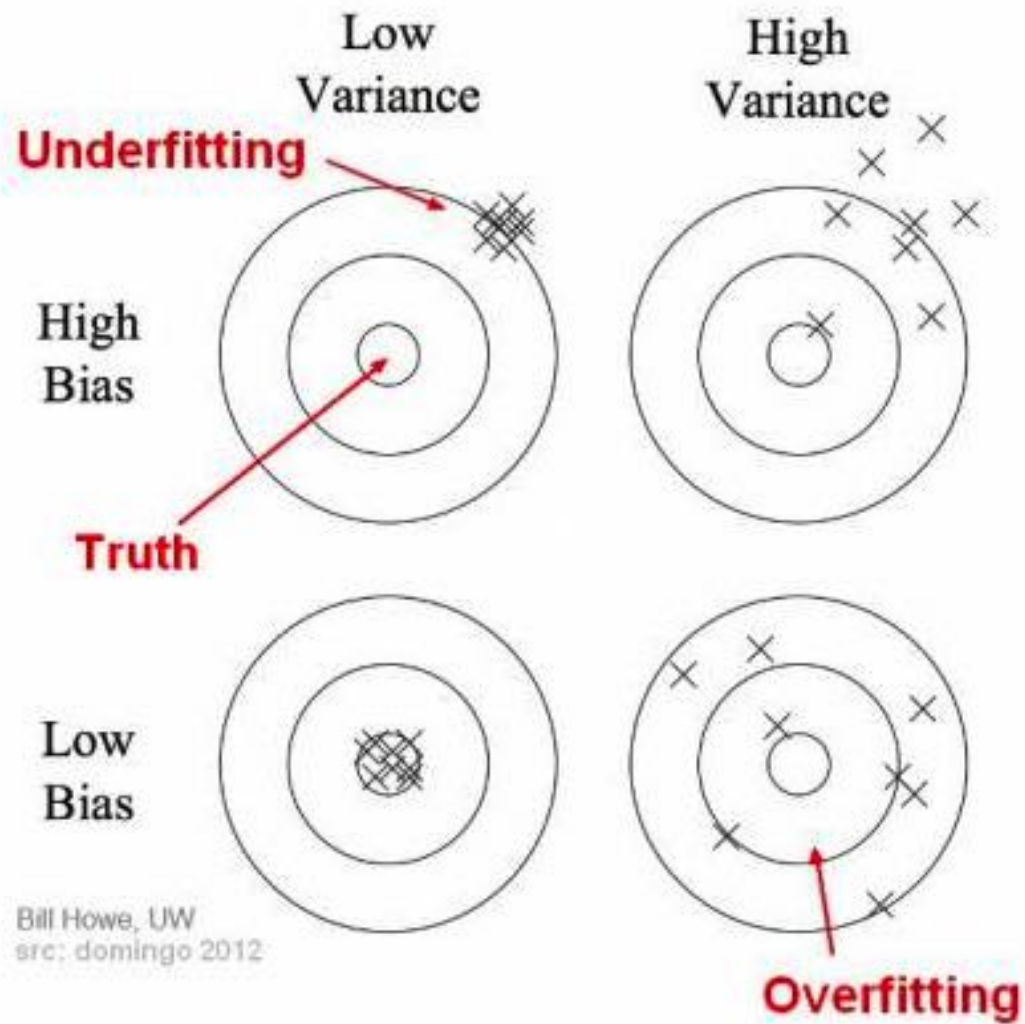


Machine Learning:

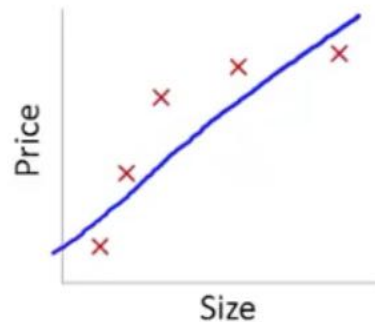


Data로부터 Specific 문제 해결을 위한 최적의 모델 만들기

Generalized Metric
Trade-off

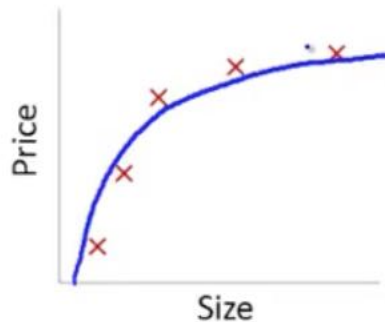


Bias/variance



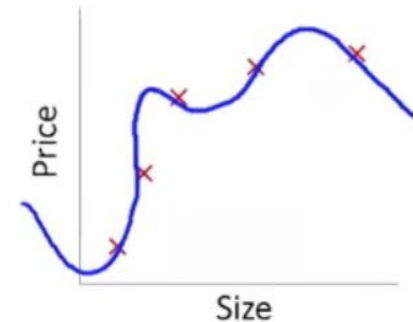
$$\theta_0 + \theta_1 x$$

High bias
(underfit)



$$\theta_0 + \theta_1 x + \theta_2 x^2$$

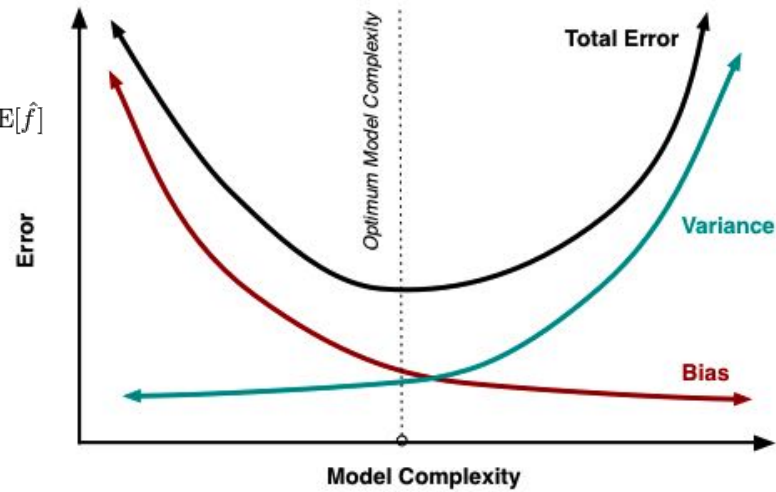
“Just right”



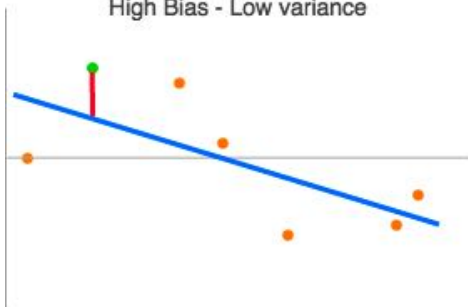
$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

High variance
(overfit)

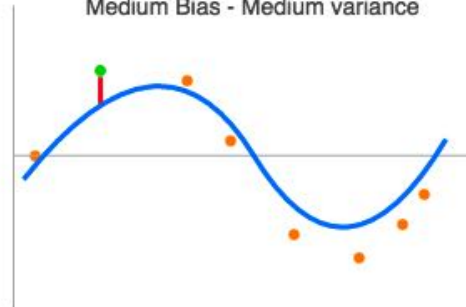
$$\begin{aligned}
 E[(y - \hat{f})^2] &= E[y^2 + \hat{f}^2 - 2y\hat{f}] \\
 &= E[y^2] + E[\hat{f}^2] - E[2y\hat{f}] \\
 &= \text{Var}[y] + E[y]^2 + \text{Var}[\hat{f}] + E[\hat{f}]^2 - 2fE[\hat{f}] \\
 &= \text{Var}[y] + \text{Var}[\hat{f}] + (f - E[\hat{f}])^2 \\
 &= \text{Var}[y] + \text{Var}[\hat{f}] + E[f - \hat{f}]^2 \\
 &= \sigma^2 + \text{Var}[\hat{f}] + \text{Bias}[\hat{f}]^2
 \end{aligned}$$



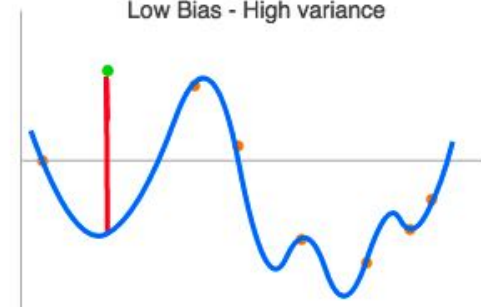
Underfit
High Bias - Low variance

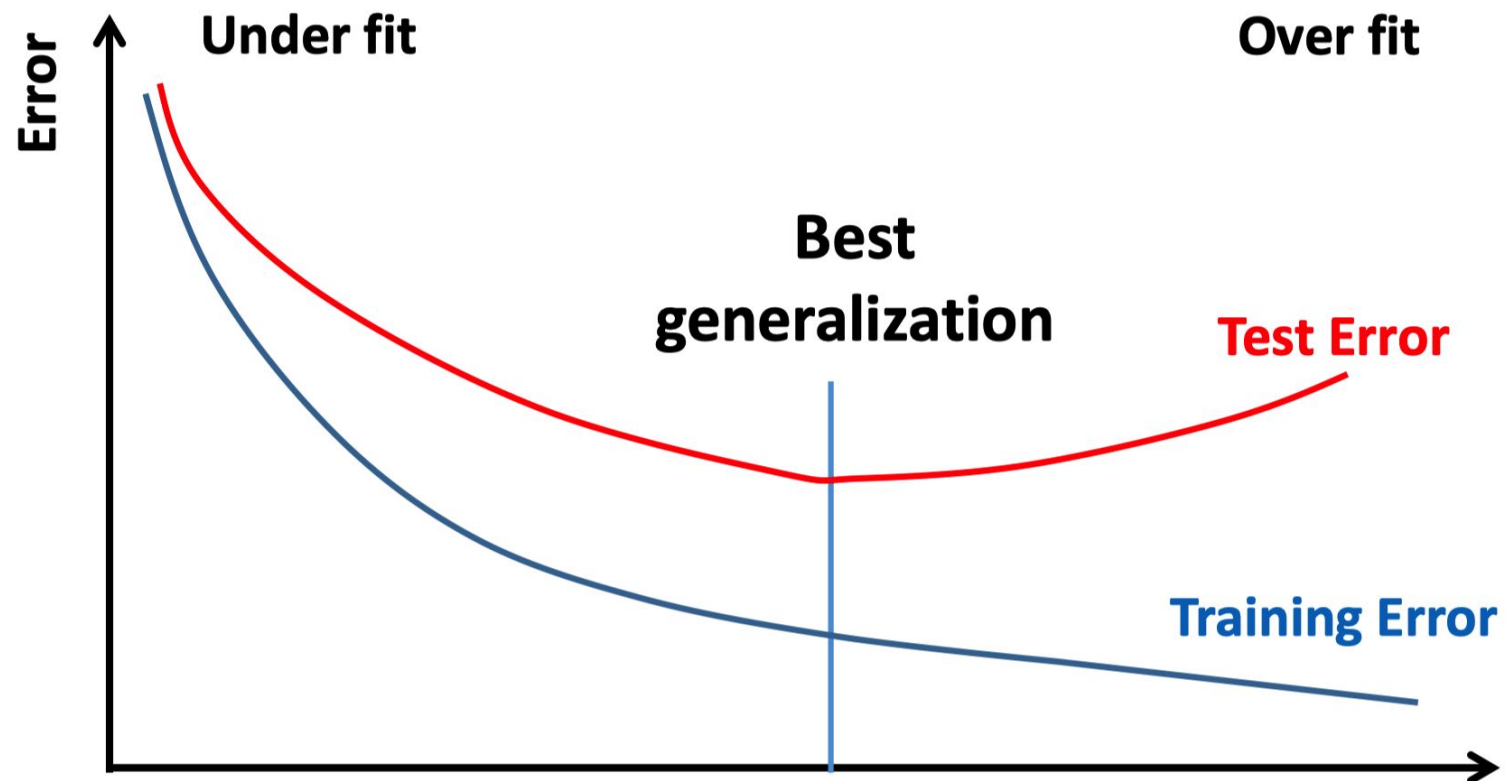


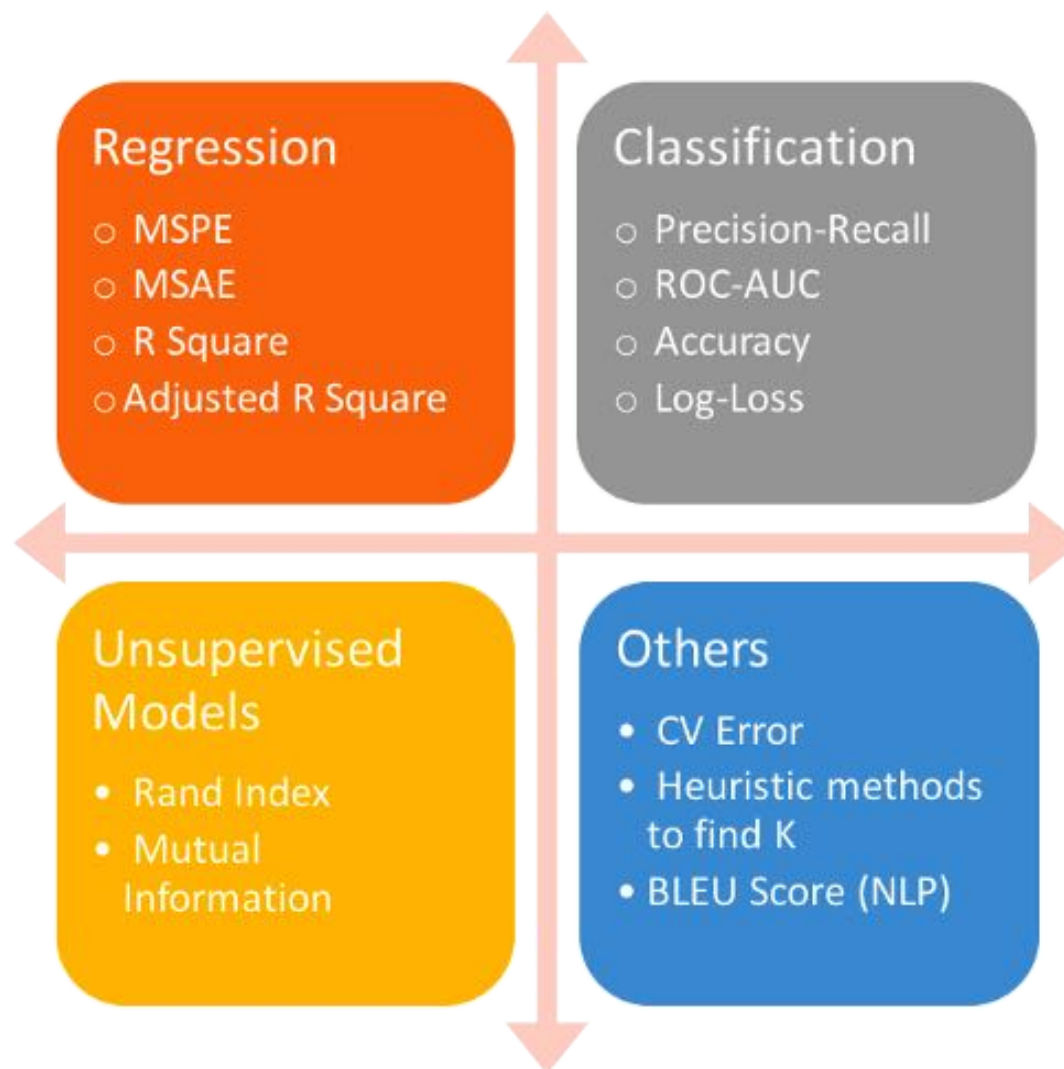
Trade-off fit
Medium Bias - Medium variance



Overfit
Low Bias - High variance





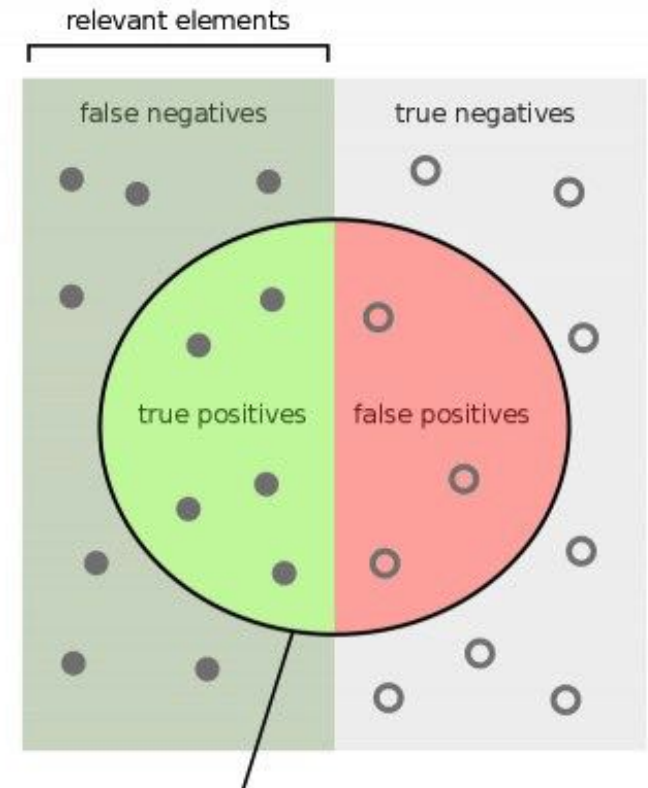


	Actual = Yes	Actual = No
Predicted = Yes	TP	FP
Predicted = No	FN	TN

Predicted

	True	False
True	True Positives	False Negatives
False	False Positives	True Negatives

Actual



selected elements

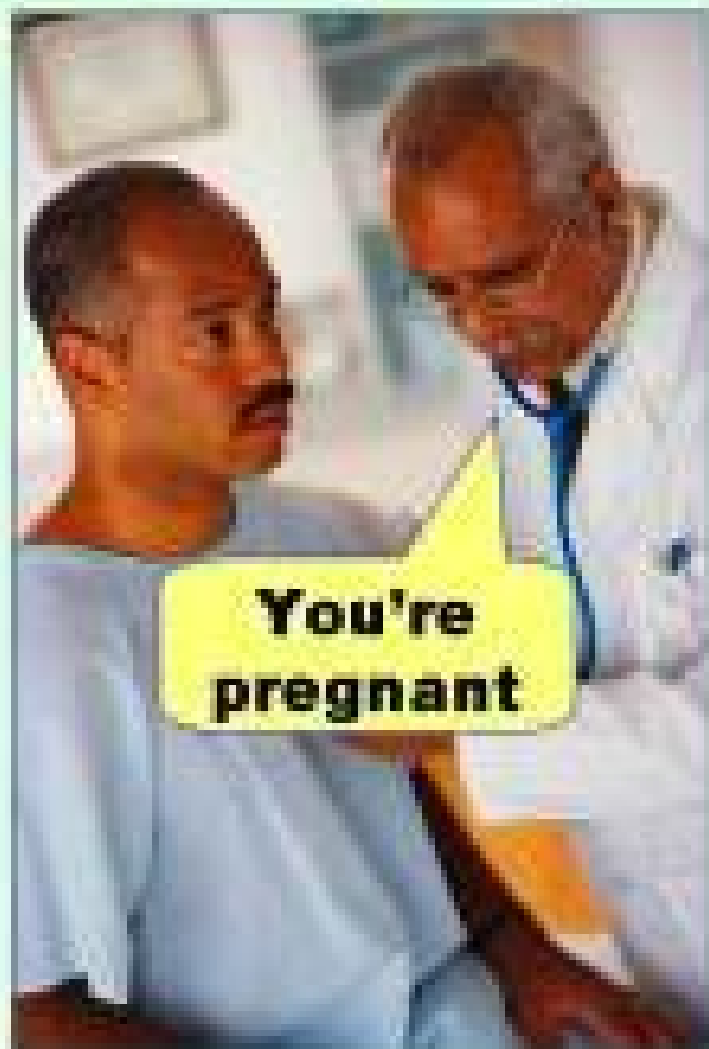
How many relevant items are selected?
e.g. How many sick people are correctly identified as having the condition.

How many negative selected elements are truly negative?
e.g. How many healthy people are identified as not having the condition.

$$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

$$\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$$

Type I error
(false positive)



Type II error
(false negative)



■ Training data improvement

- Get more training examples (Fixes over fitting)
- Ensure training data is high quality (De-noise training data)

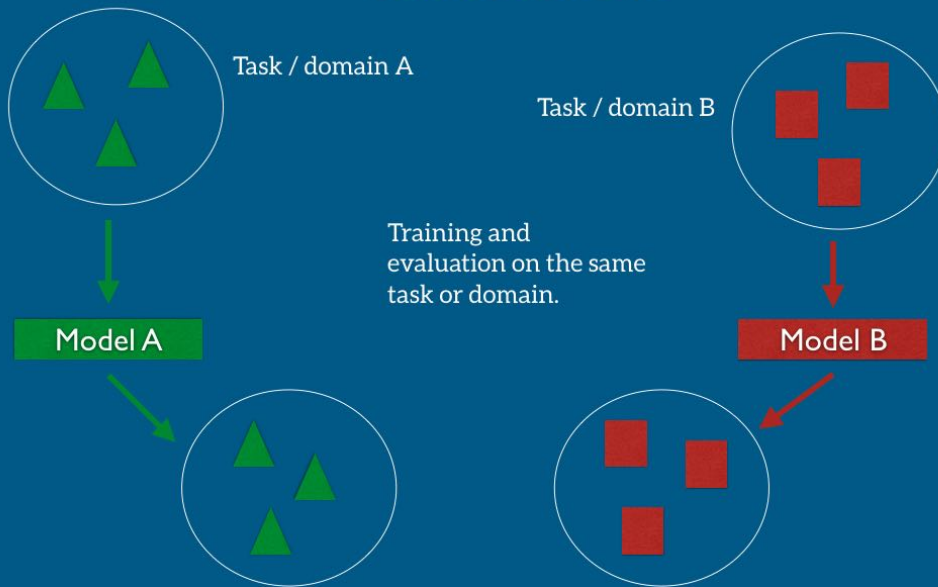
■ Feature engineering

- Increase/reduce number of features (Fixes under/over fitting)
- Change features used (Fixes under fitting)

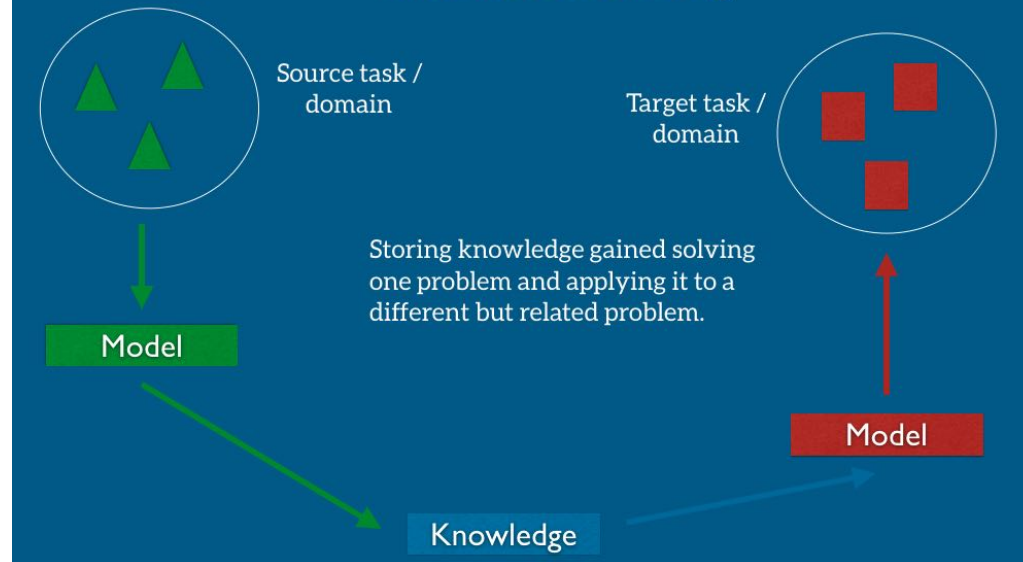
■ Optimization algorithm

- Change the ML model used (SVMs , Decision trees, neural network, etc.)
- Run optimization algo for more iterations to ensure it converges

Traditional ML



Transfer learning



1. Workflow

Machine Learning Workflow

Asking
the right
question

Preparing
data

Selecting
the
algorithm

Training
the
model

Testing
the
model



Early steps are most important

Each step depends on previous steps

Expect to go backwards

Later knowledge effects previous steps

Data is never as you need it

Data will have to be altered

More data is better

More Data => Better Results

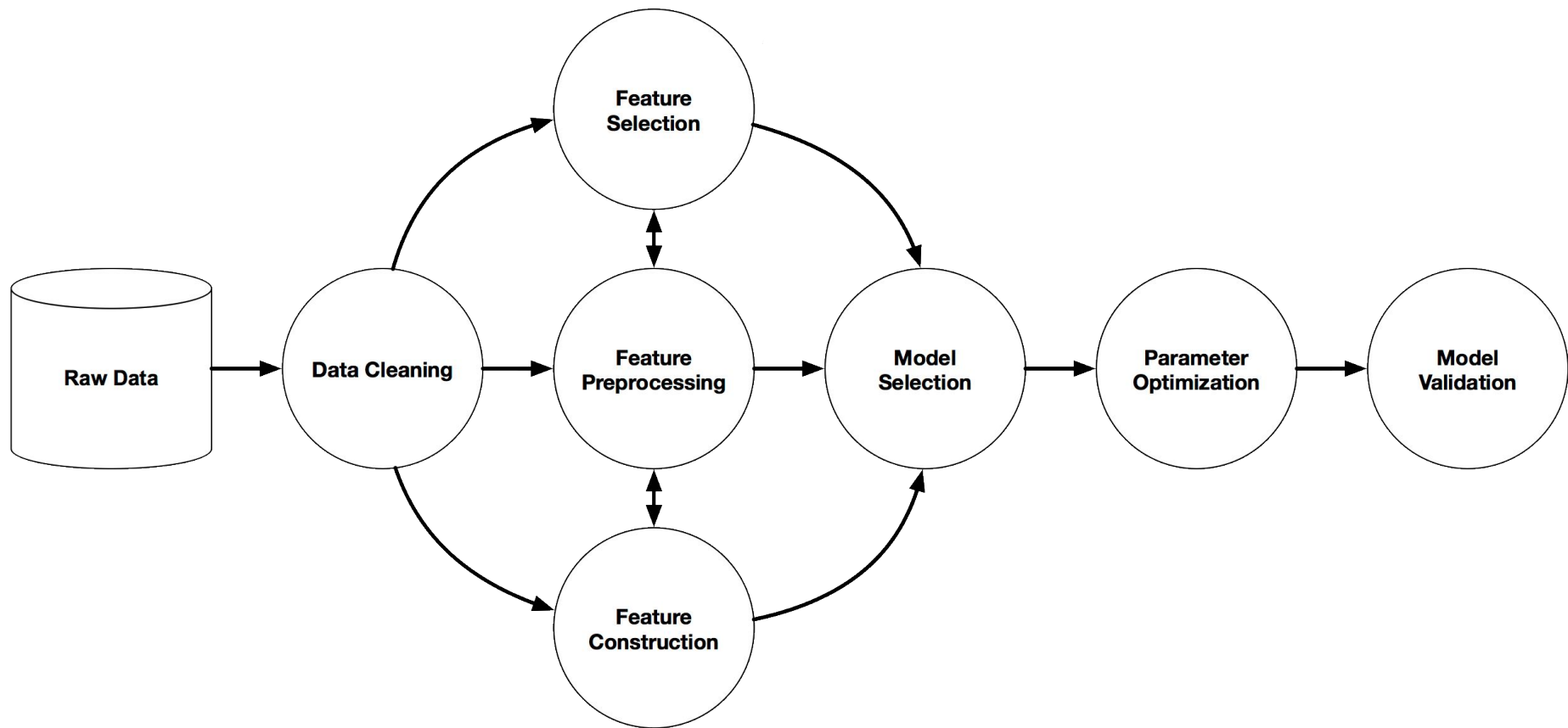
Think before acting

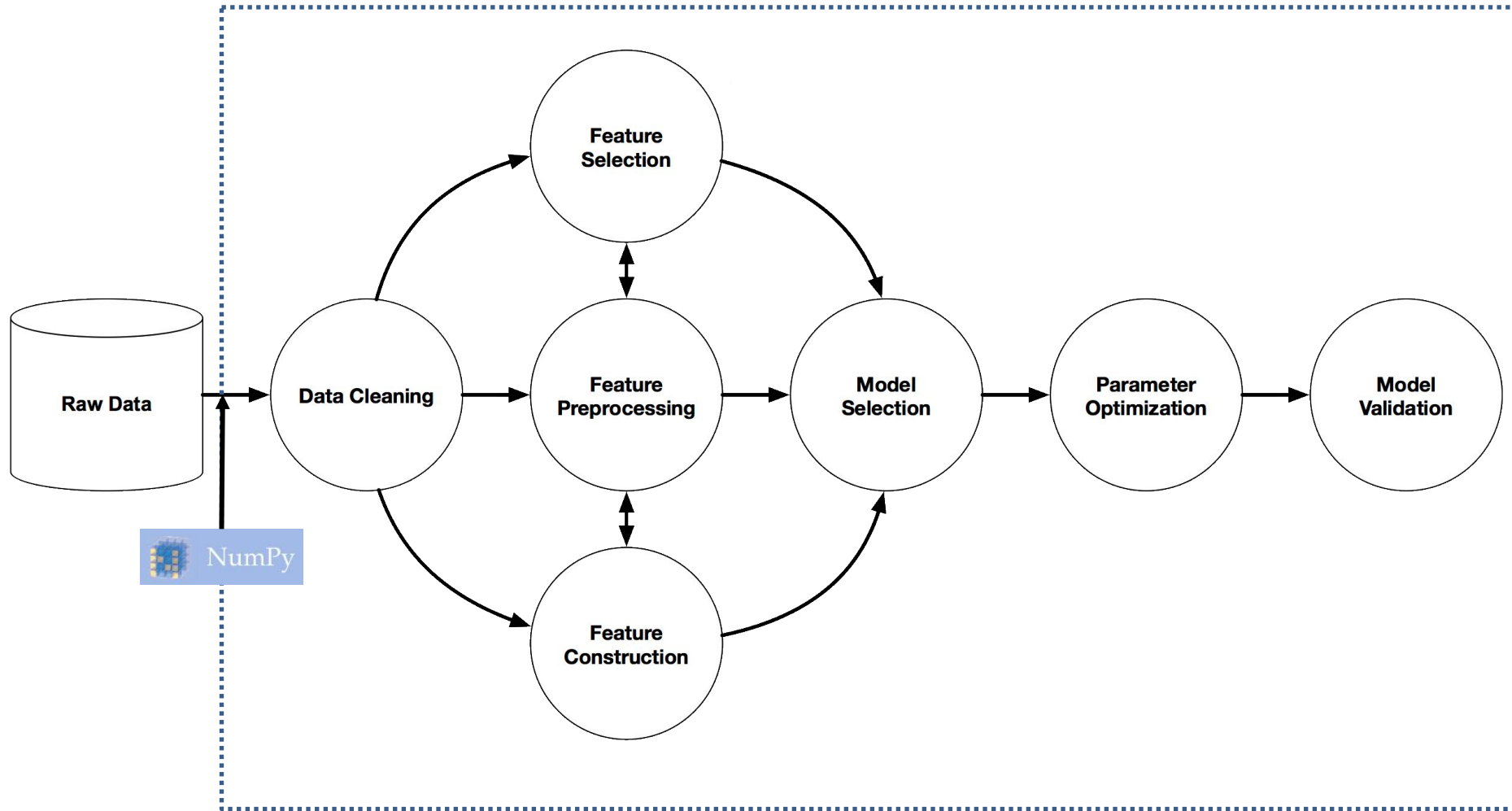
Understand results before changing data or model

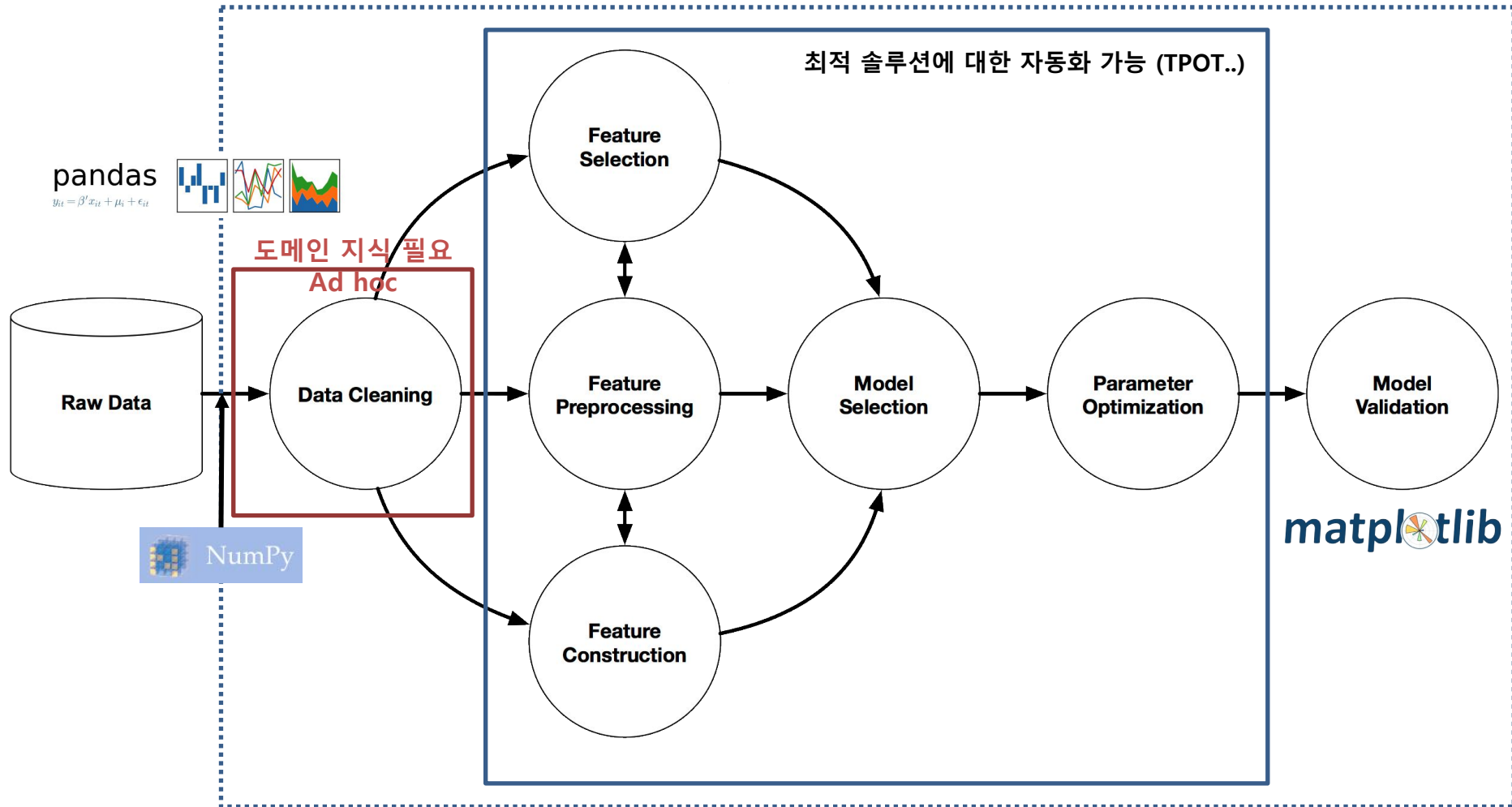
Don't pursue a bad solution

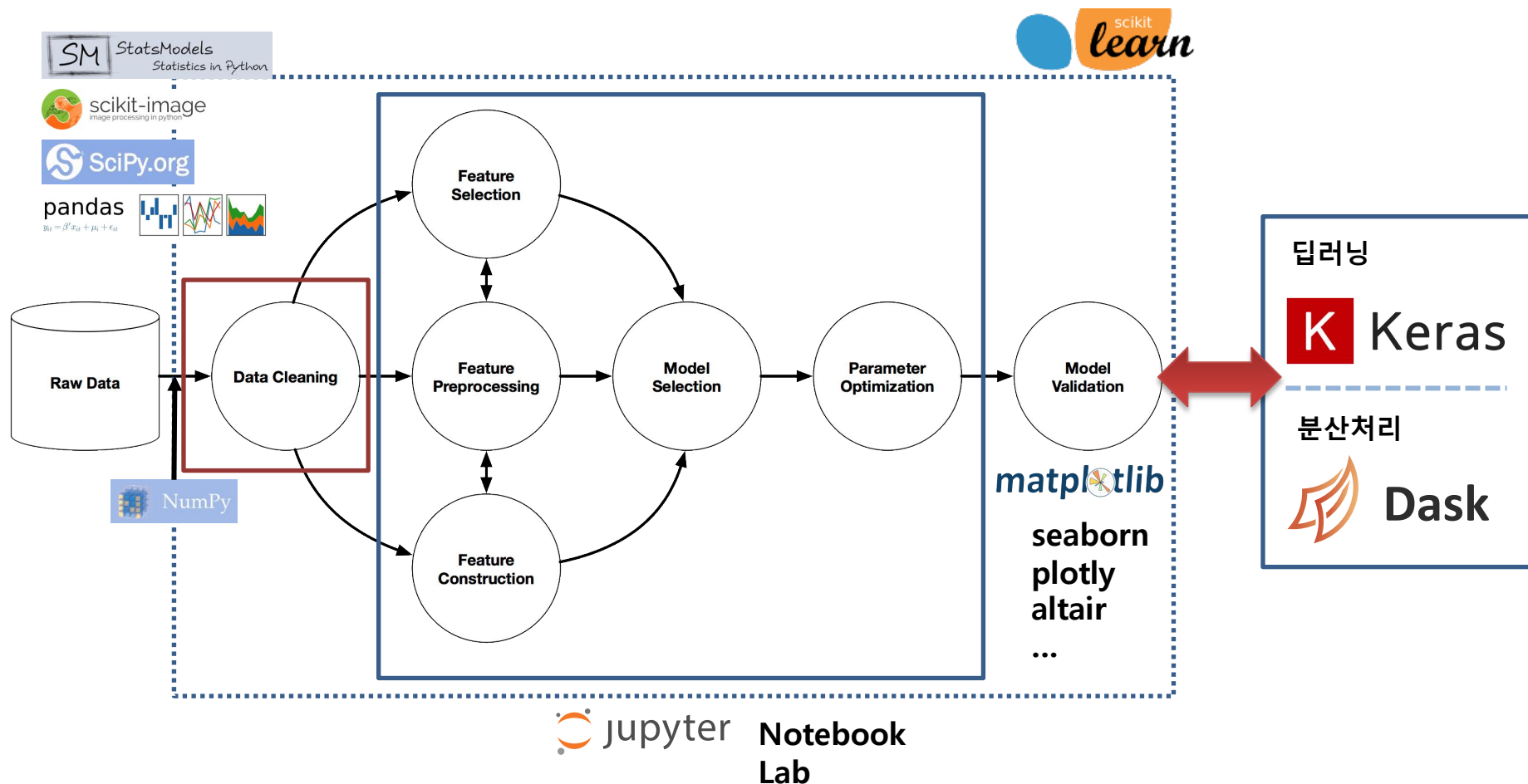
Reevaluate, fix or quit

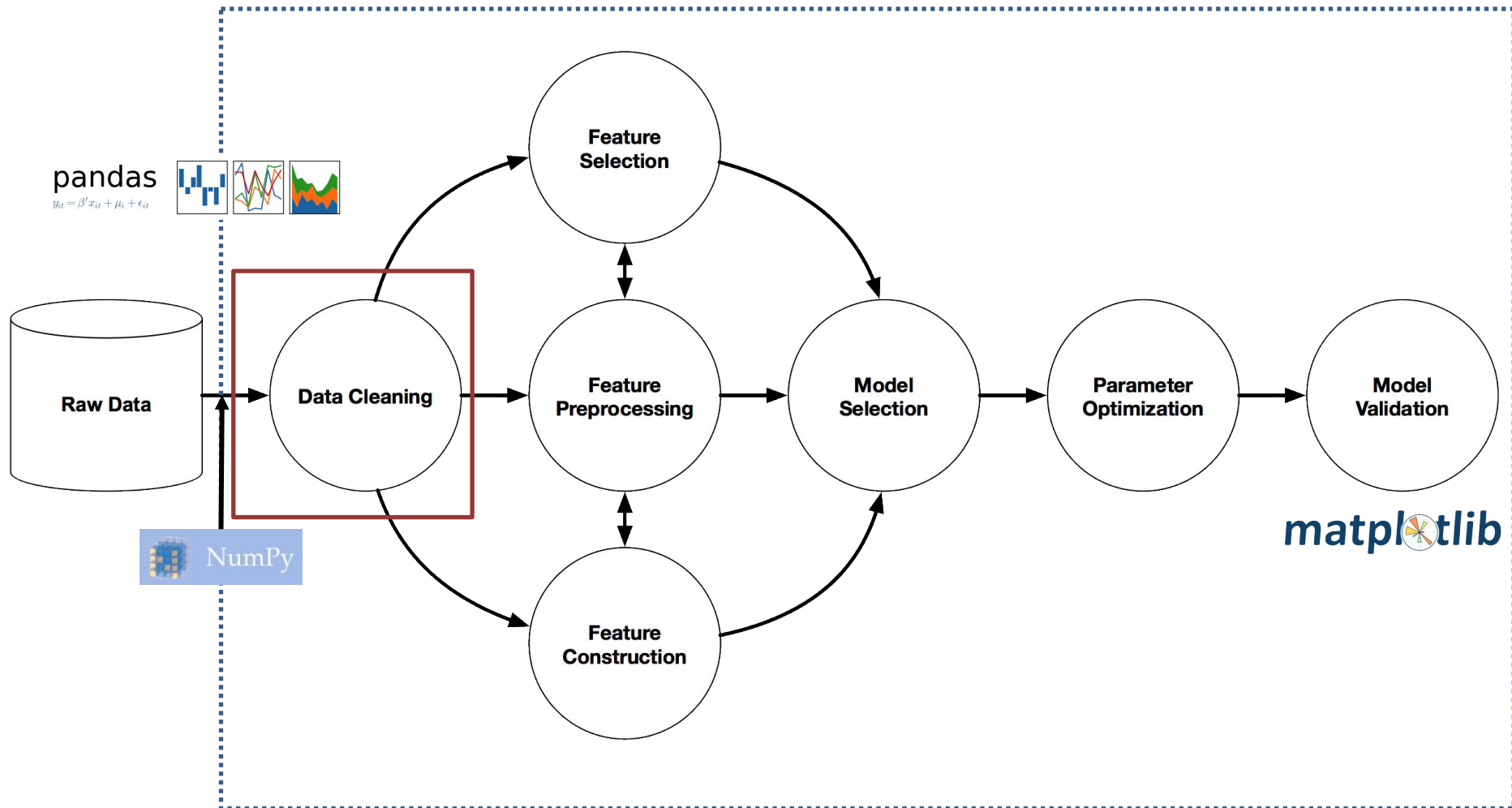
- 0. Define the problem / question**
- I. Identify and collect data**
- II. Explore and prepare data**
- III. Build and evaluate model**
- IV. Communicate results**

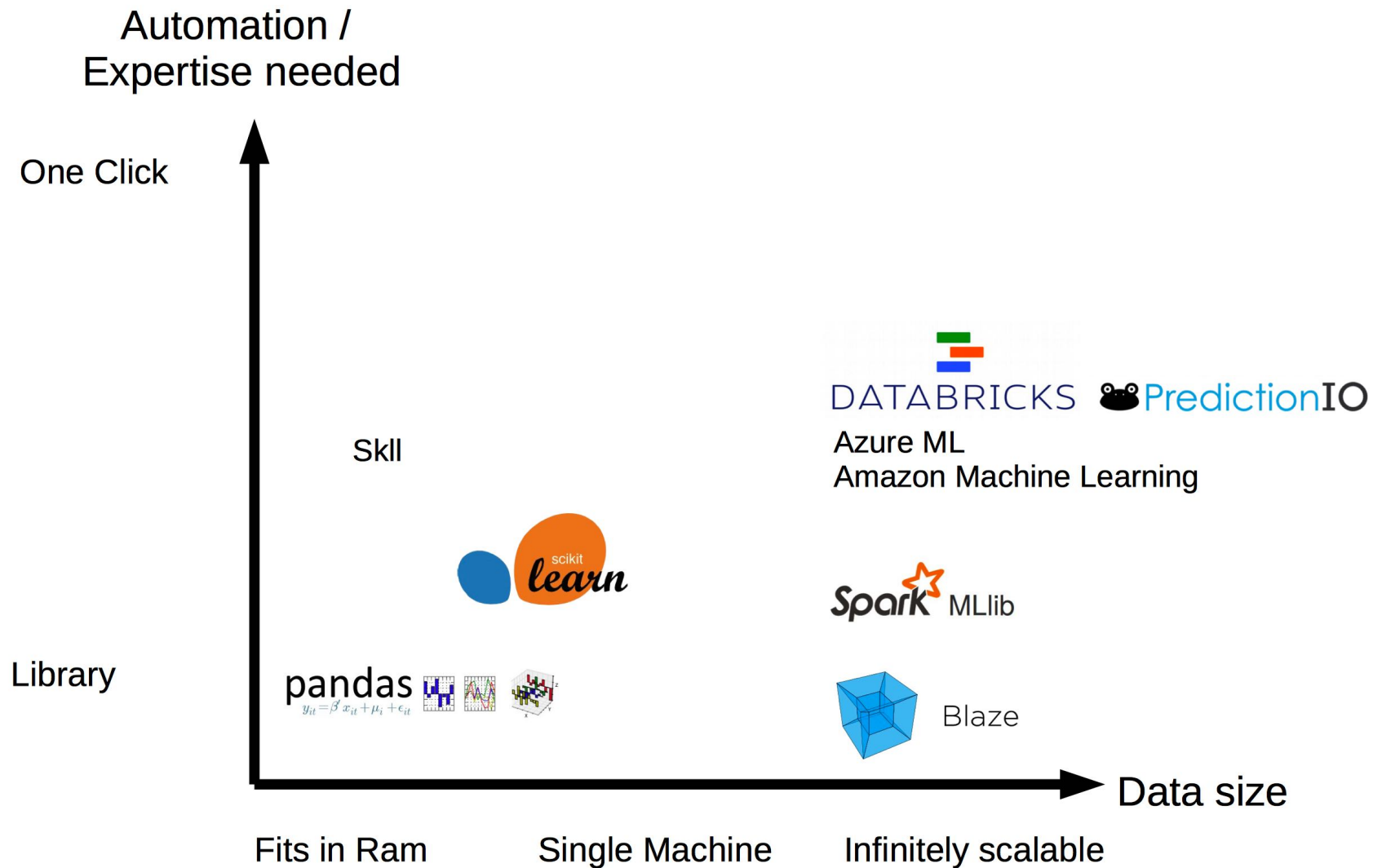


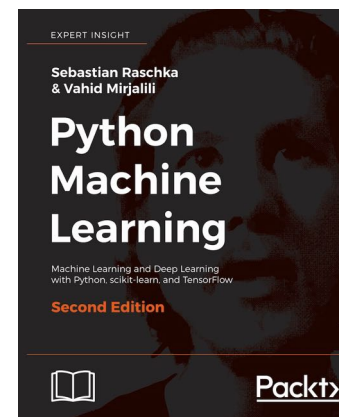
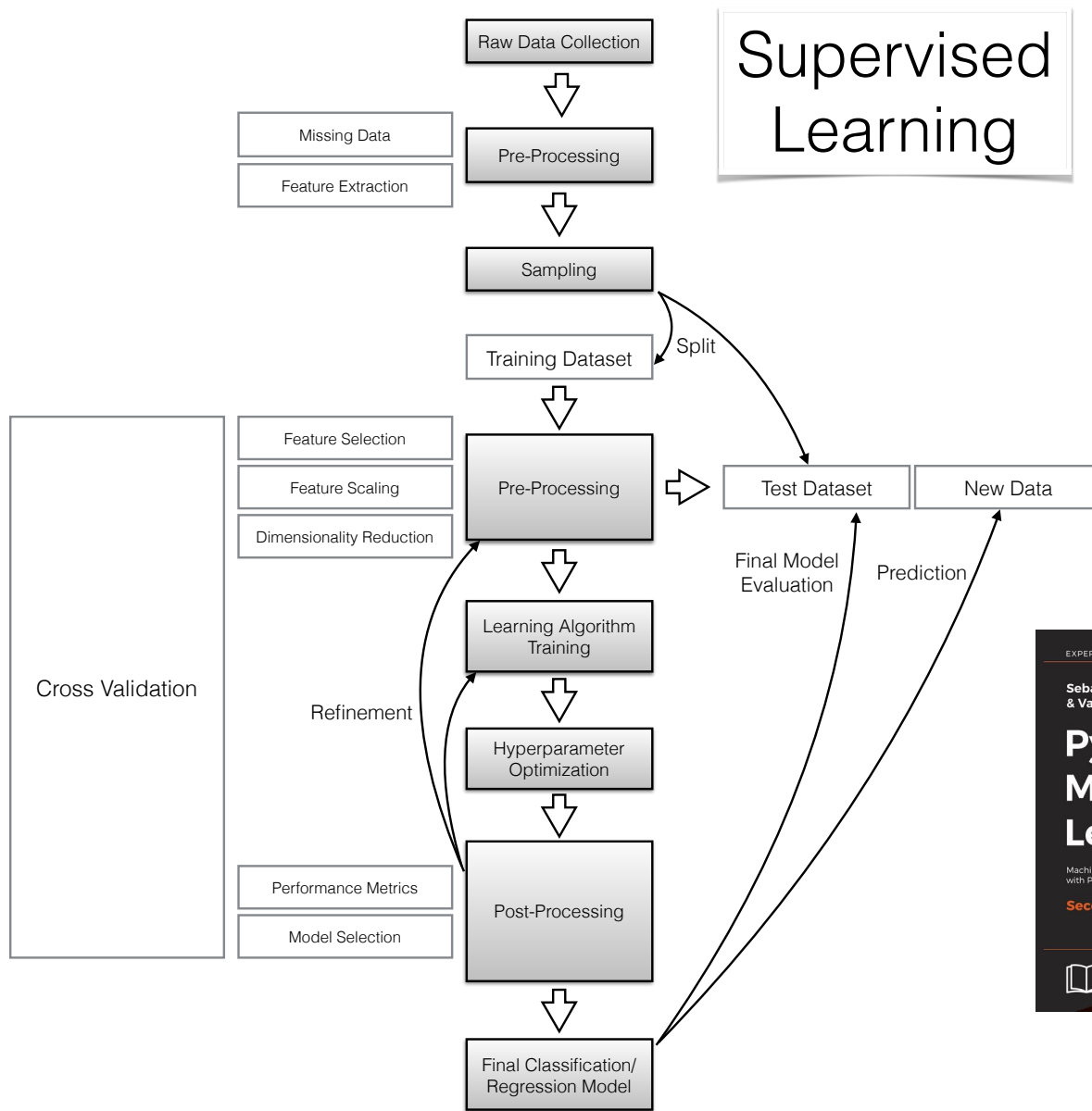












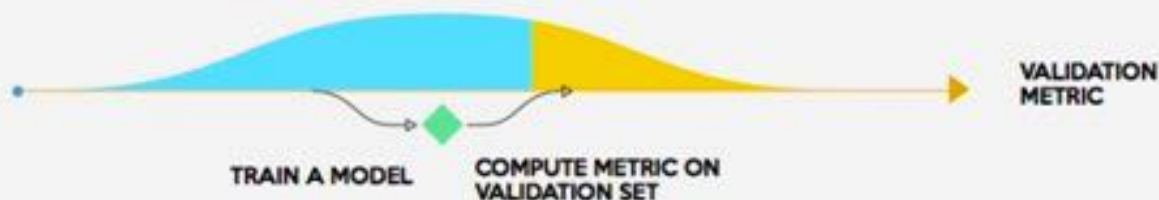
HOLDOUT STRATEGY

1 Split your data into train / validation / test



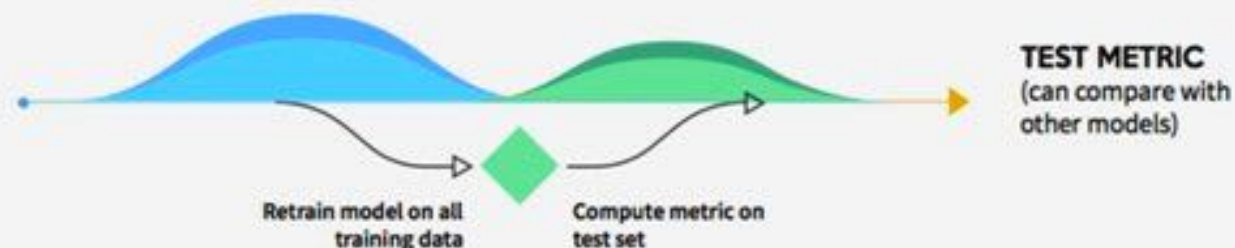
2 For each parameter combination

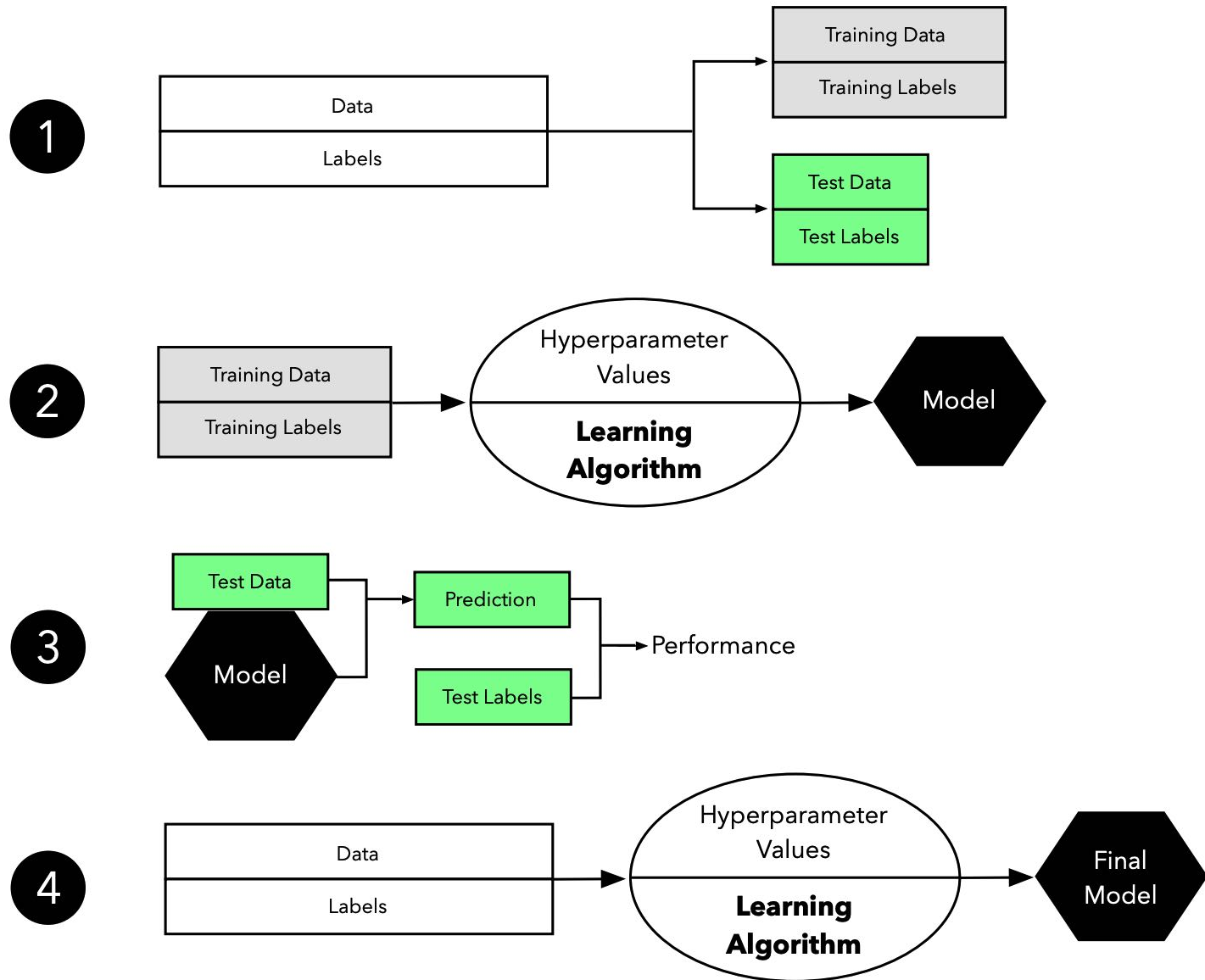
Parameter (e.g., depth) A	4	11	Parameter B (e.g., n trees)
	5	15	
	6	16	
	7	17	

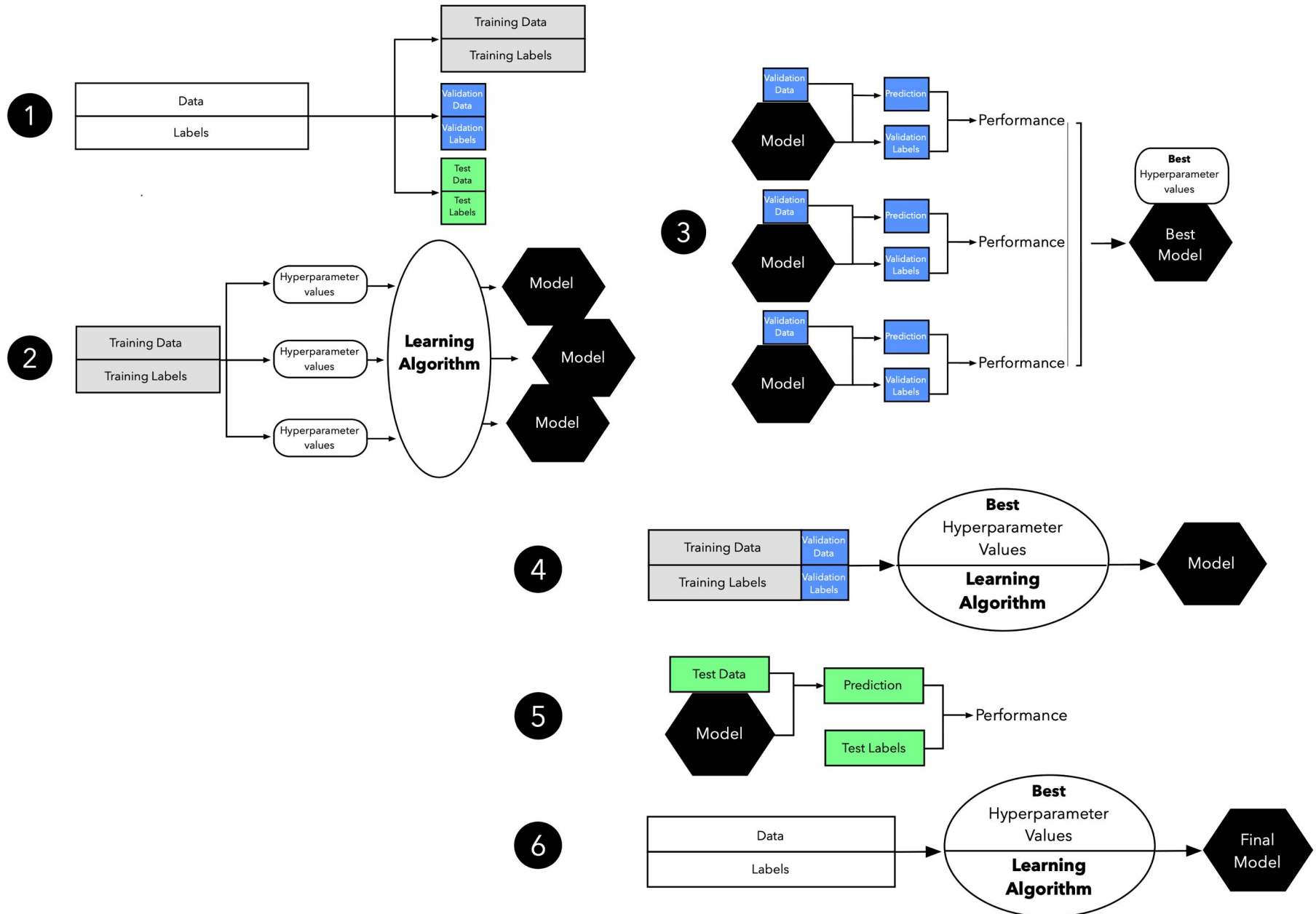


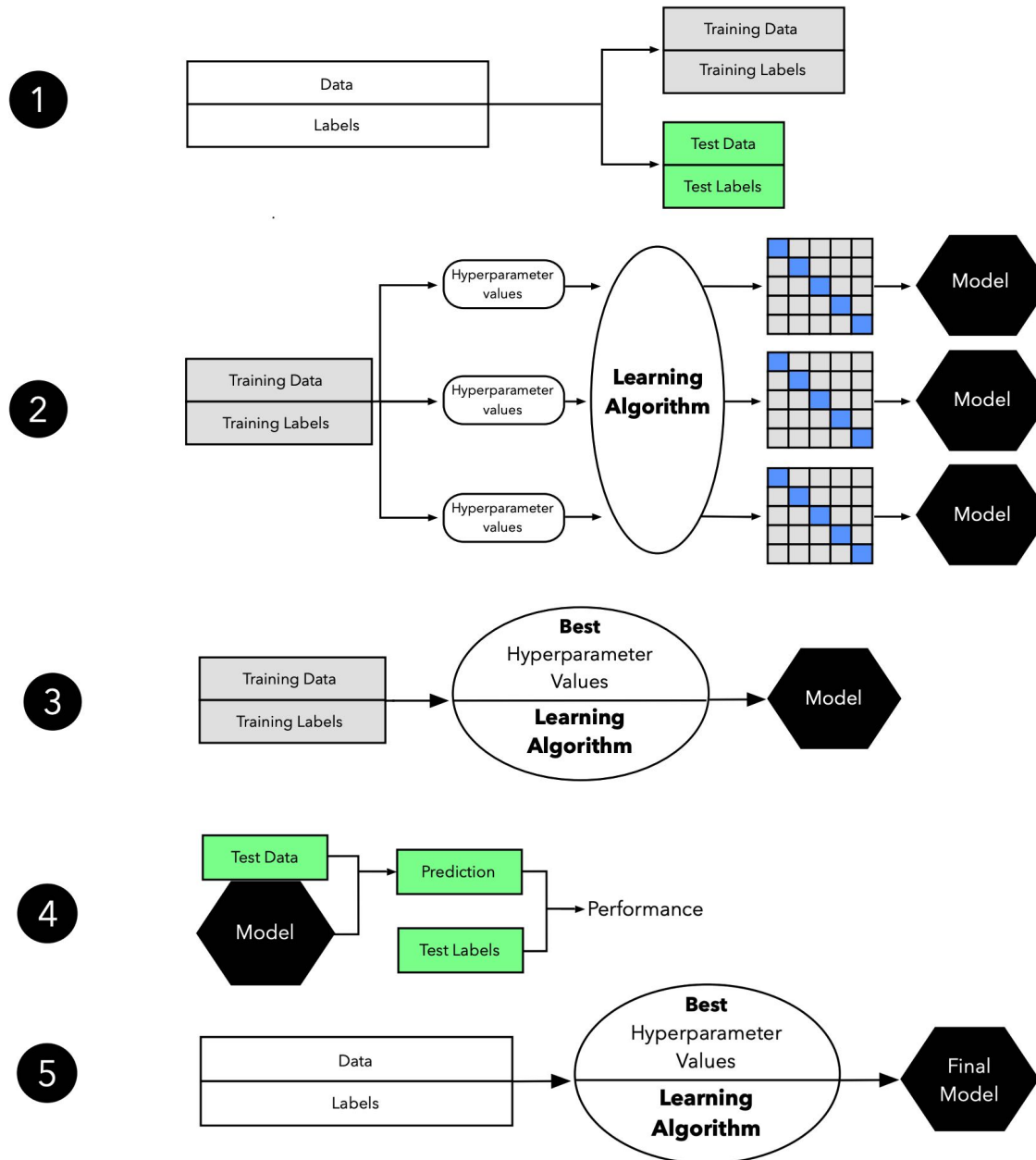
3 Choose the parameter combination with the best metric

Parameter (e.g., depth) A	6	14	Parameter B (e.g., n trees)
------------------------------	---	----	--------------------------------



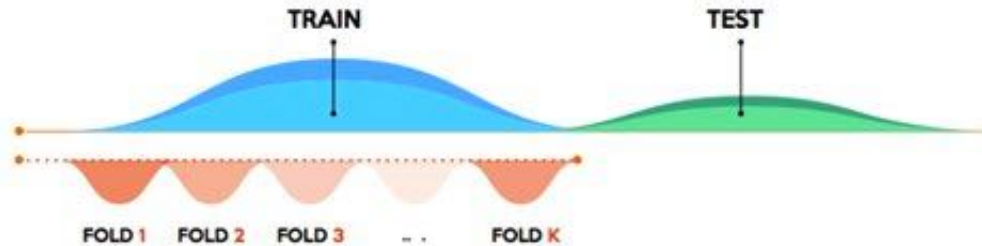






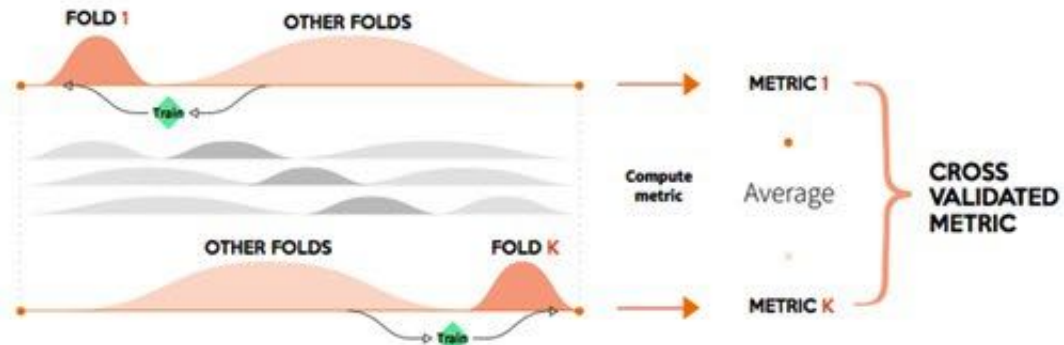
K-FOLD STRATEGY

1 Set aside the test set and split the train set into k folds



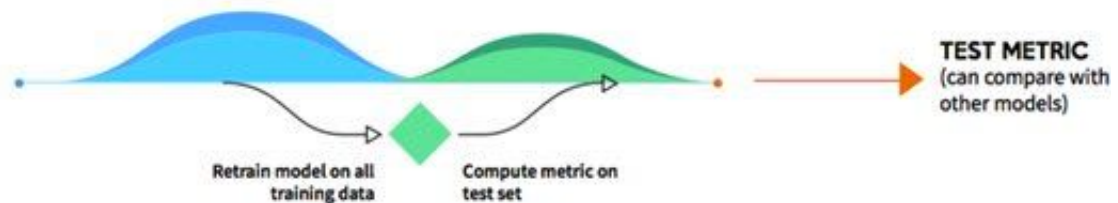
2 For each parameter combination

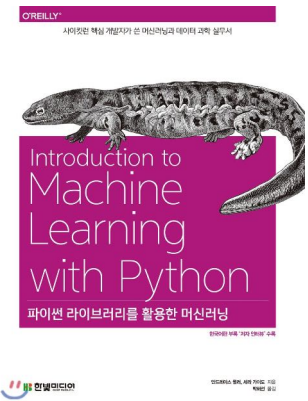
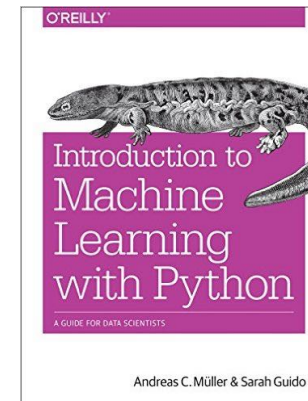
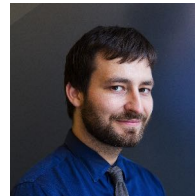
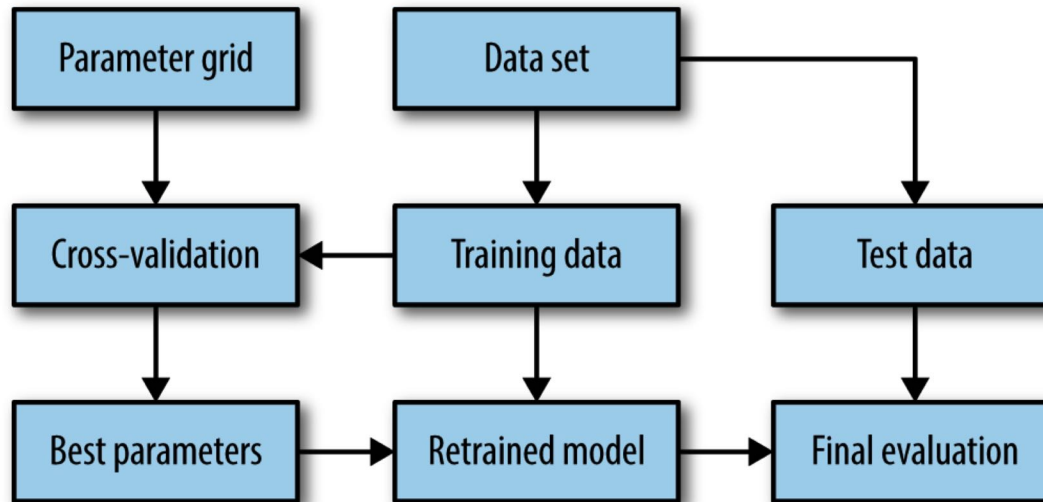
Parameter (e.g., depth)	A	1	11	Parameter B (e.g., n trees)
		5	15	
		6	16	
		7	17	

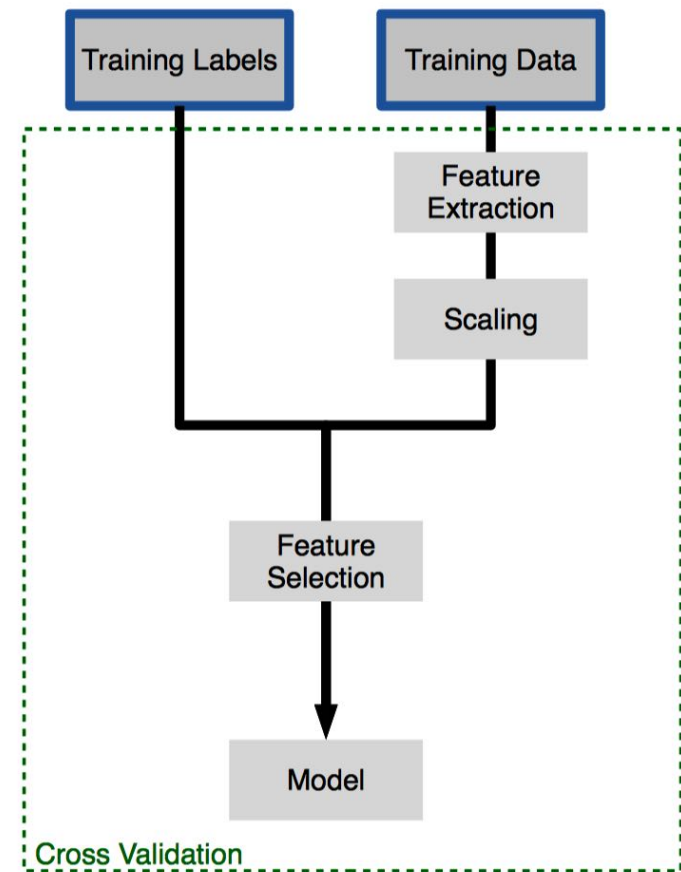
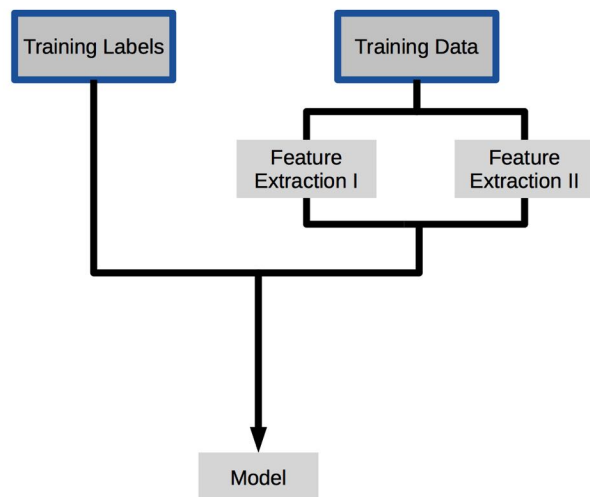
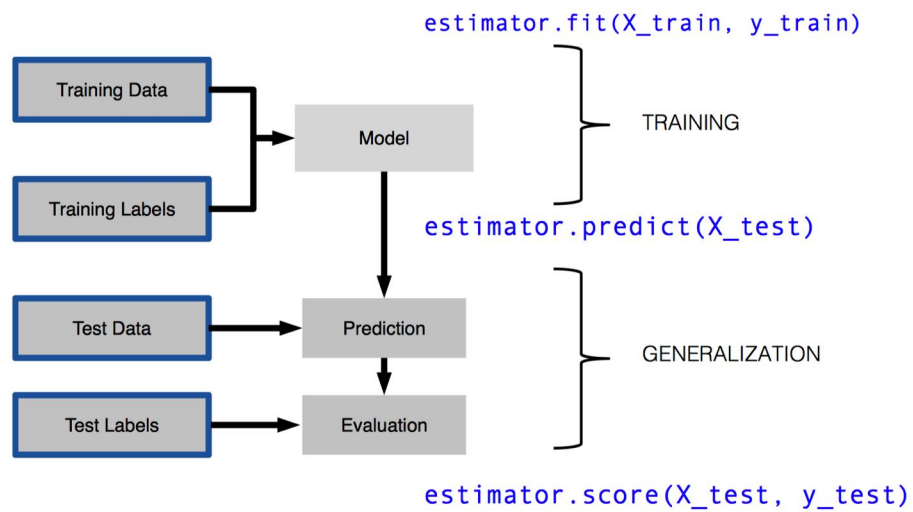


3 Choose the parameter combination with the best metrics

Parameter (e.g., depth)	A	6	14	Parameter B (e.g., n trees)
-------------------------	---	---	----	-----------------------------



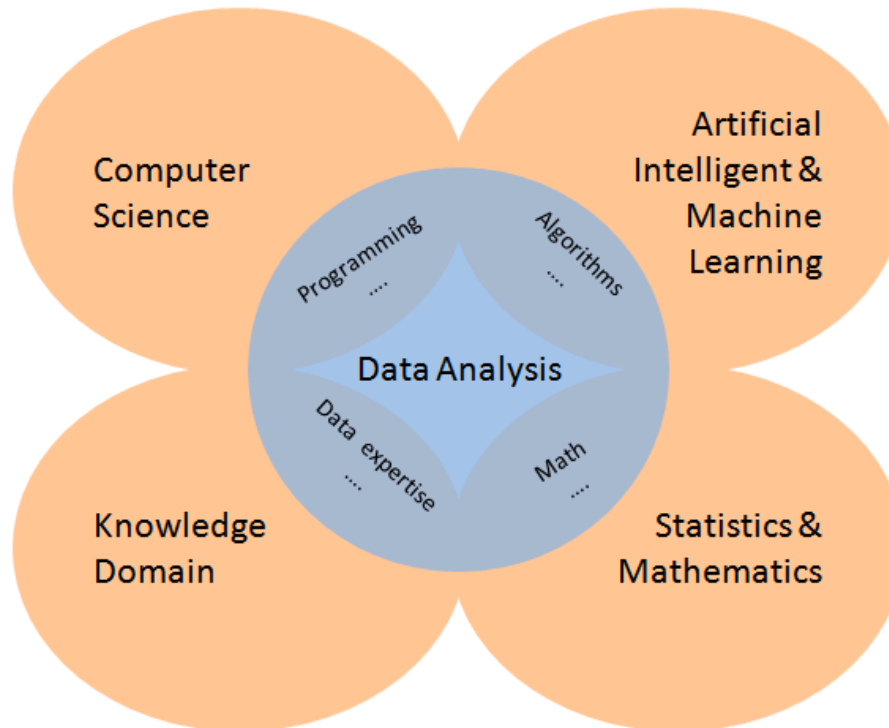




2. Data

■ Combine human intuition/ wisdom with machine speed/ pattern recognition.

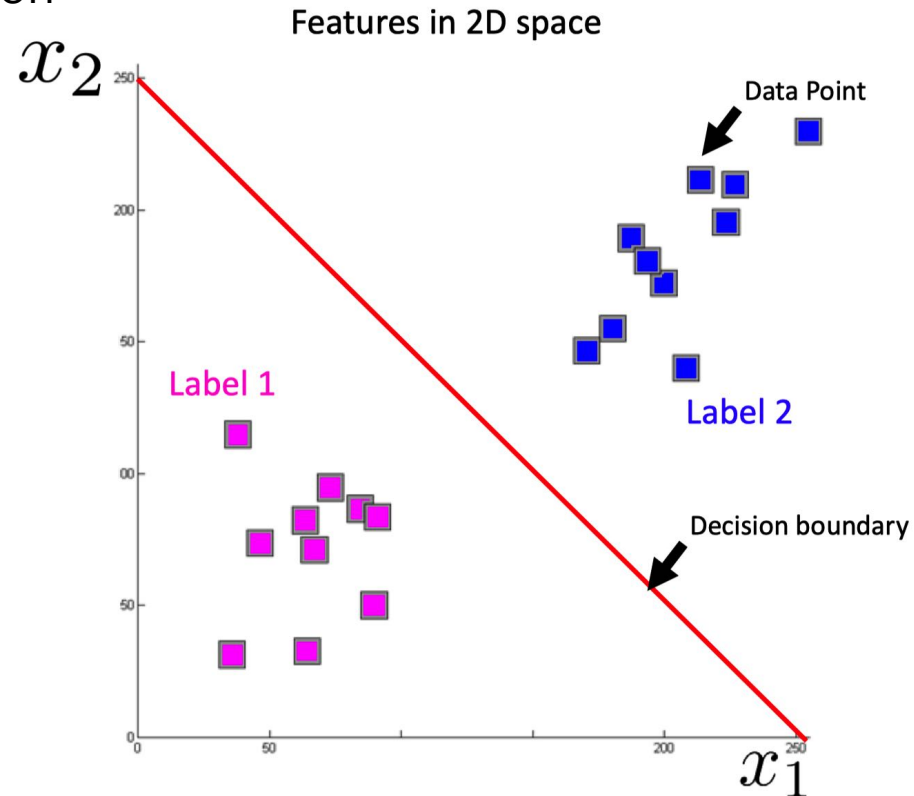
- Use domain knowledge to choose features that distinguishes different classes well
- Add specific rules to process input data before training stage/ output data after test stage. (In contrast with generalized rules from machine learning) Reduces training time when data is pre processed instead of letting ML model learn the patterns eventually.



- Data point = Sample = Example = observation
- Labels/ Classes/ Categories:
 - Discrete (e.g. Optical Character Recognition)
 - Continuous (e.g. Housing prices)
- Classification/decision boundary:
 - Separates regions of feature space
 - Hopefully helps separate different classes

$V =$

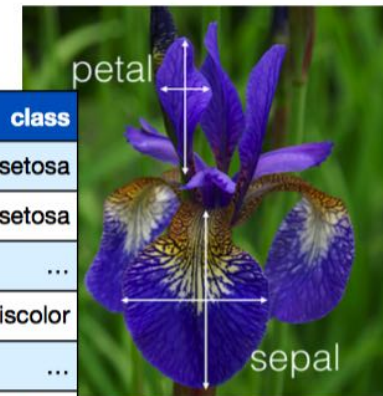
	Feature dimension j	Label
Index i of data point		



<https://archive.ics.uci.edu/ml/datasets/Iris>

Instances (samples, observations)

	sepal_length	sepal_width	petal_length	petal_width	class
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
...	...	...	...	...	...
50	6.4	3.2	4.5	1.5	vericolor
...	...	...	...	...	...
150	5.9	3.0	5.1	1.8	virginica

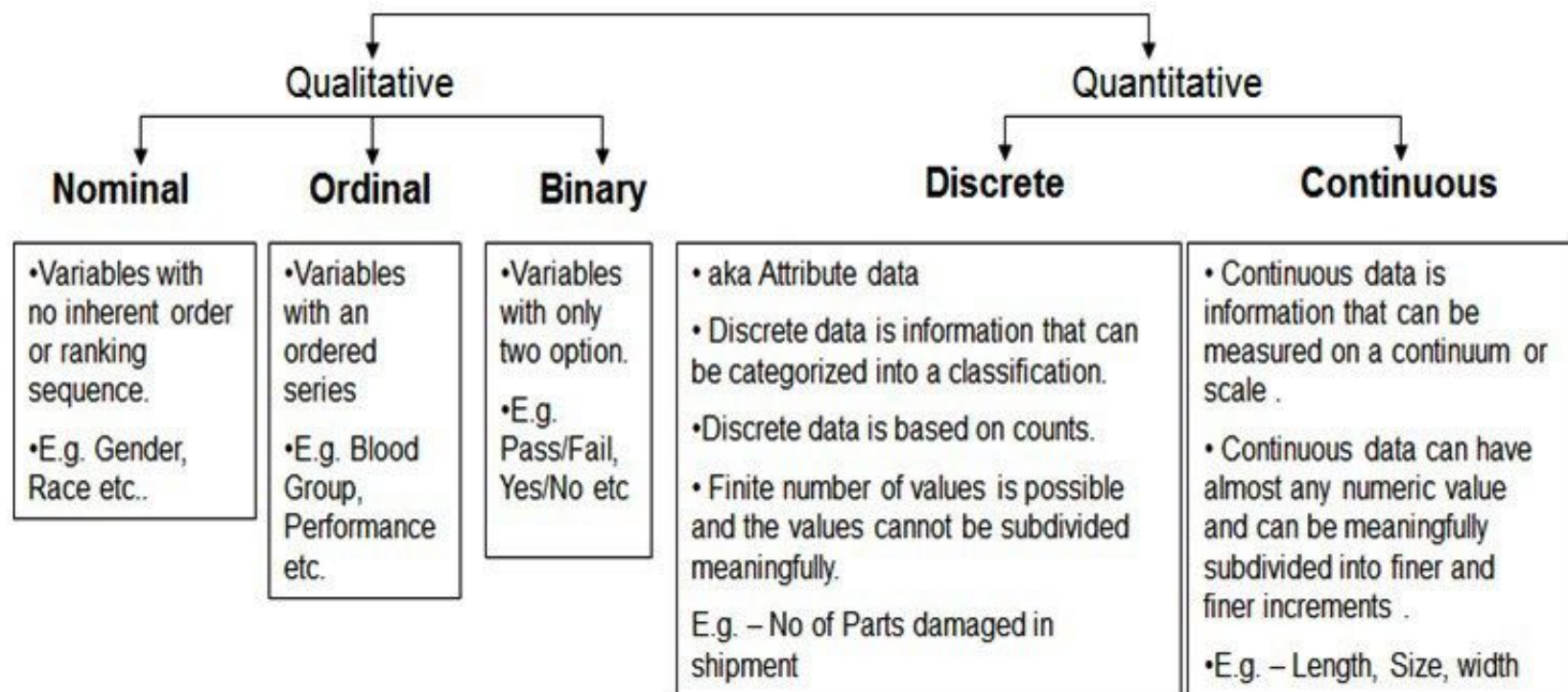


Features (attributes, dimensions)

Classes (targets) labels

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1D} \\ x_{21} & x_{22} & \cdots & x_{2D} \\ x_{31} & x_{32} & \cdots & x_{3D} \\ \vdots & \vdots & \ddots & \vdots \\ \vdots & \vdots & \ddots & \vdots \\ x_{N1} & x_{N2} & \cdots & x_{ND} \end{bmatrix}$$

$$\mathbf{y} = [y_1, y_2, y_3, \cdots y_N]$$



	color	size	prize	class
0	green	M	10.1	class1
1	red	L	13.5	class2
2	blue	XL	15.3	class1

nominalgreen $\rightarrow (1,0,0)$ red $\rightarrow (0,1,0)$ blue $\rightarrow (0,0,1)$ **ordinal**M $\rightarrow 1$ L $\rightarrow 2$ XL $\rightarrow 3$

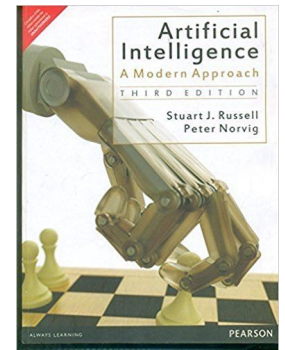
	class	color=blue	color=green	color=red	prize	size
0	0	0	1	0	10.1	1
1	1	0	0	1	13.5	2
2	0	1	0	0	15.3	3

Big data		Small Data	
No overfitting to training data		Overfitting to training data is possible	
Working with complex models for difficult problems		Working with simple models for easy problems	
Higher computational cost → Slower learning time		Lower computation cost → Faster learning time	
Large-scale storage is required		Small-scale storage is fine	

Raw data	Featured Data
No information loss → Discover Unrecognized patterns	Some information loss → Degrade performance sometime
Curse of dimensionality	Dimensionality Reduction
Higher computational cost → Slower learning time	Lower computation cost → Faster learning time
Large-scale storage is required	Small-scale storage is fine

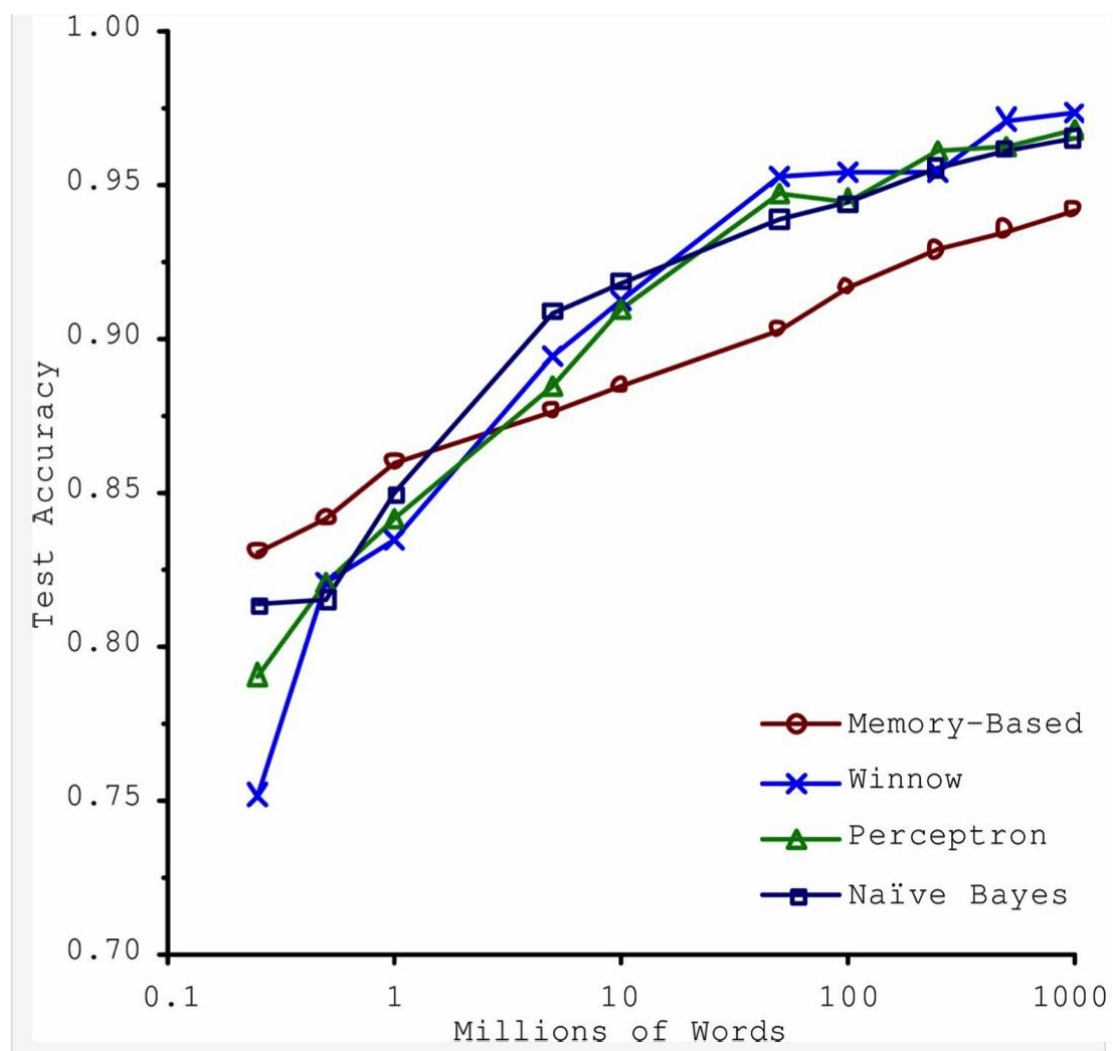
The Unreasonable Effectiveness of Data (2009)

Peter Norvig



Learning Curves for Confusion Set Disambiguation (2001)

Michele Banko & Eric Brill



How Much Information is the Machine Given during Learning?

► “Pure” Reinforcement Learning (**cherry**)

- The machine predicts a scalar reward given once in a while.

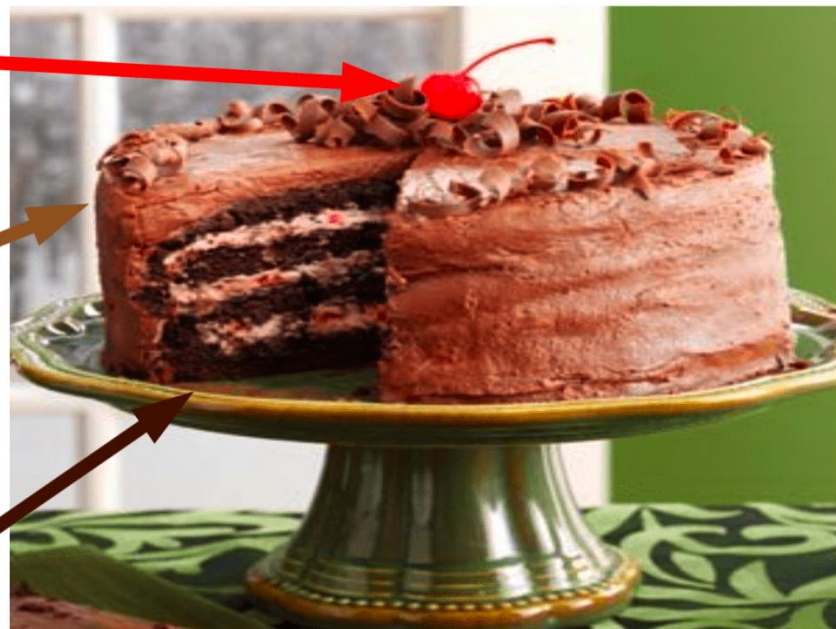
► **A few bits for some samples**

► Supervised Learning (**icing**)

- The machine predicts a category or a few numbers for each input
- Predicting human-supplied data
- **10→10,000 bits per sample**

► Self-Supervised Learning (**cake génoise**)

- The machine predicts any part of its input for any observed part.
- Predicts future frames in videos
- **Millions of bits per sample**



■ “Pure” Reinforcement Learning (cherry)

- ▶ The machine predicts a scalar reward given once in a while.
- ▶ **A few bits for some samples**

■ Supervised Learning (icing)

- ▶ The machine predicts a category or a few numbers for each input
- ▶ Predicting human-supplied data
- ▶ **10→10,000 bits per sample**

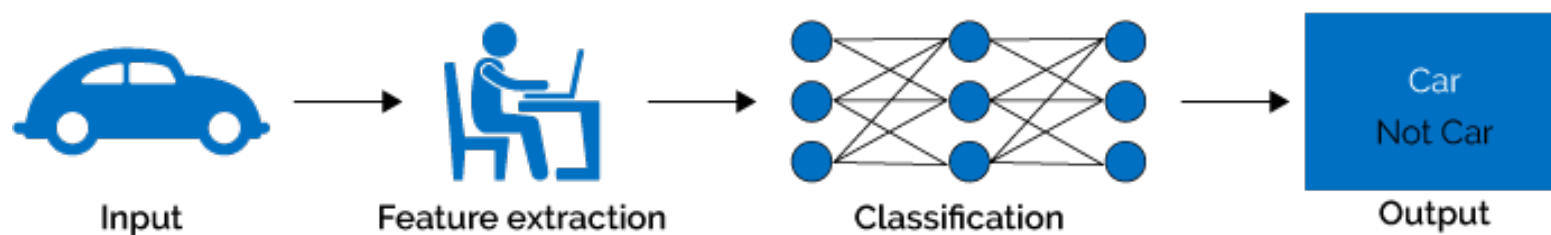
■ Unsupervised/Predictive Learning (cake)

- ▶ The machine predicts any part of its input for any observed part.
- ▶ Predicts future frames in videos
- ▶ **Millions of bits per sample**

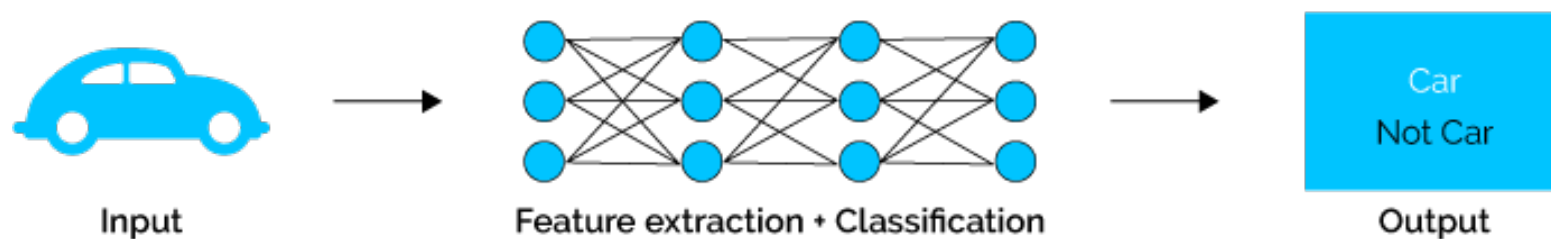


■ (Yes, I know, this picture is slightly offensive to RL folks. But I'll make it up)

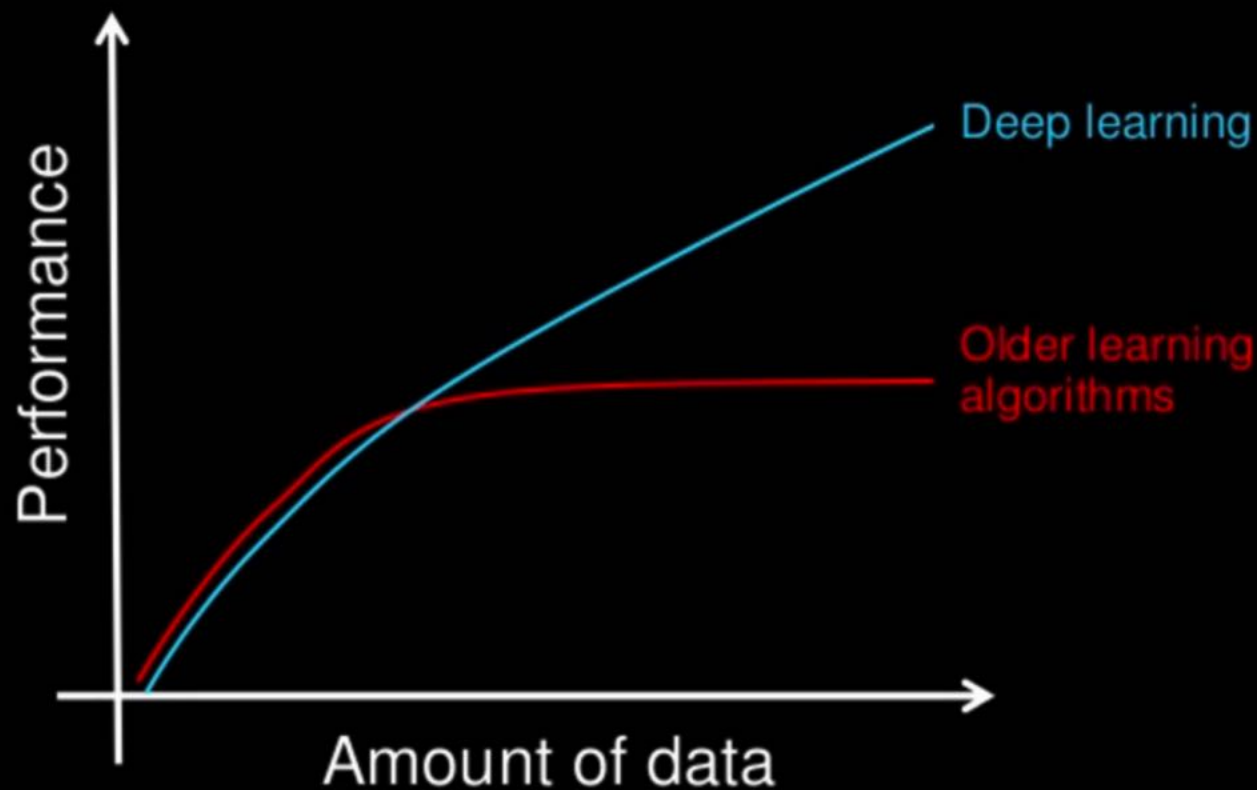
Machine Learning



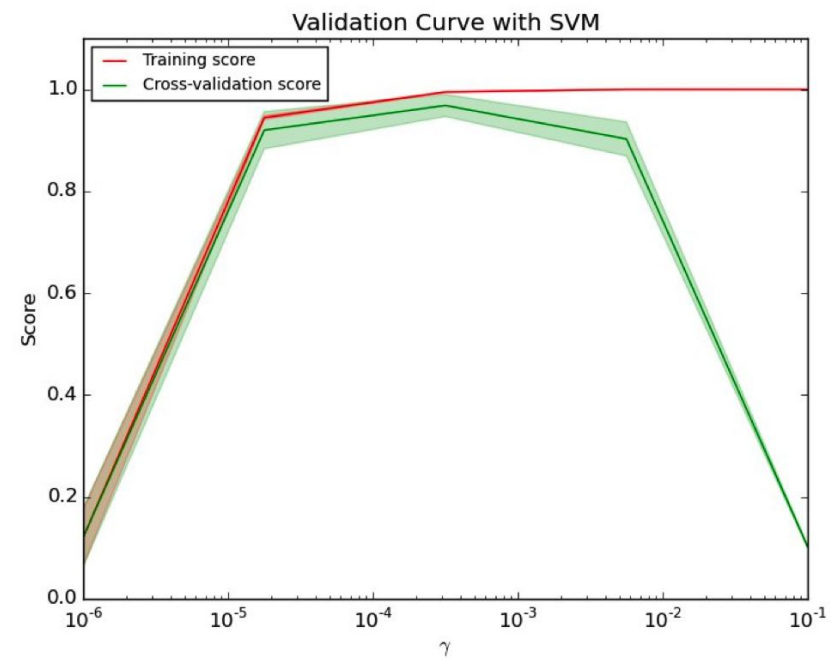
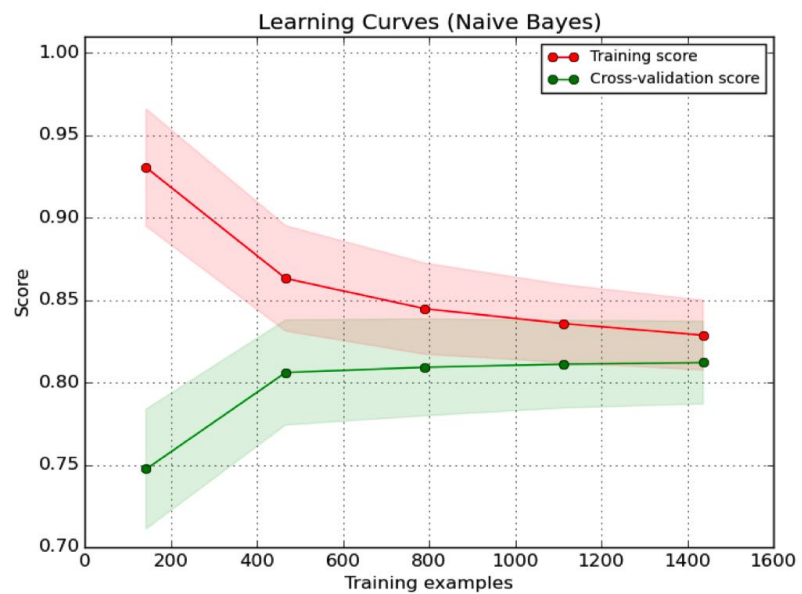
Deep Learning



Why deep learning

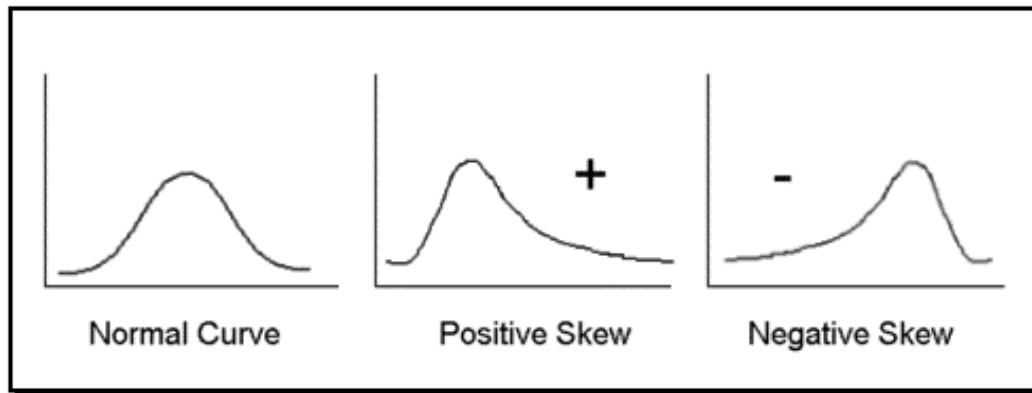


How do data science techniques scale with amount of data?



■ Data Cleaning & Preprocessing

- Handling outliers
- Handling missing values
- Handling skewness

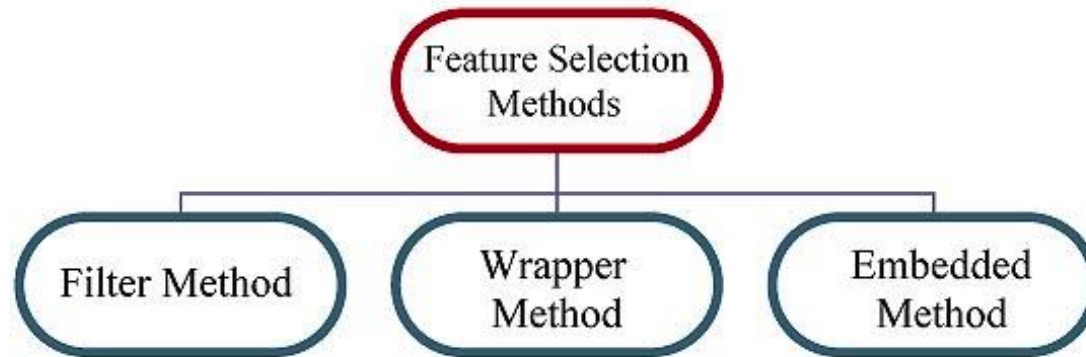


■ Scaling

■ Encoding

- One Hot Encoding
- Label Encoding

■ Feature Extraction/Selection



■ Dimension Reduction

- Linear discriminant analysis (LDA)
- Principal Component Analysis(PCA)

Missing Values:

- Remove features (columns)
- Remove samples (rows)
- Imputation (mean, nearest neighbor, ...)

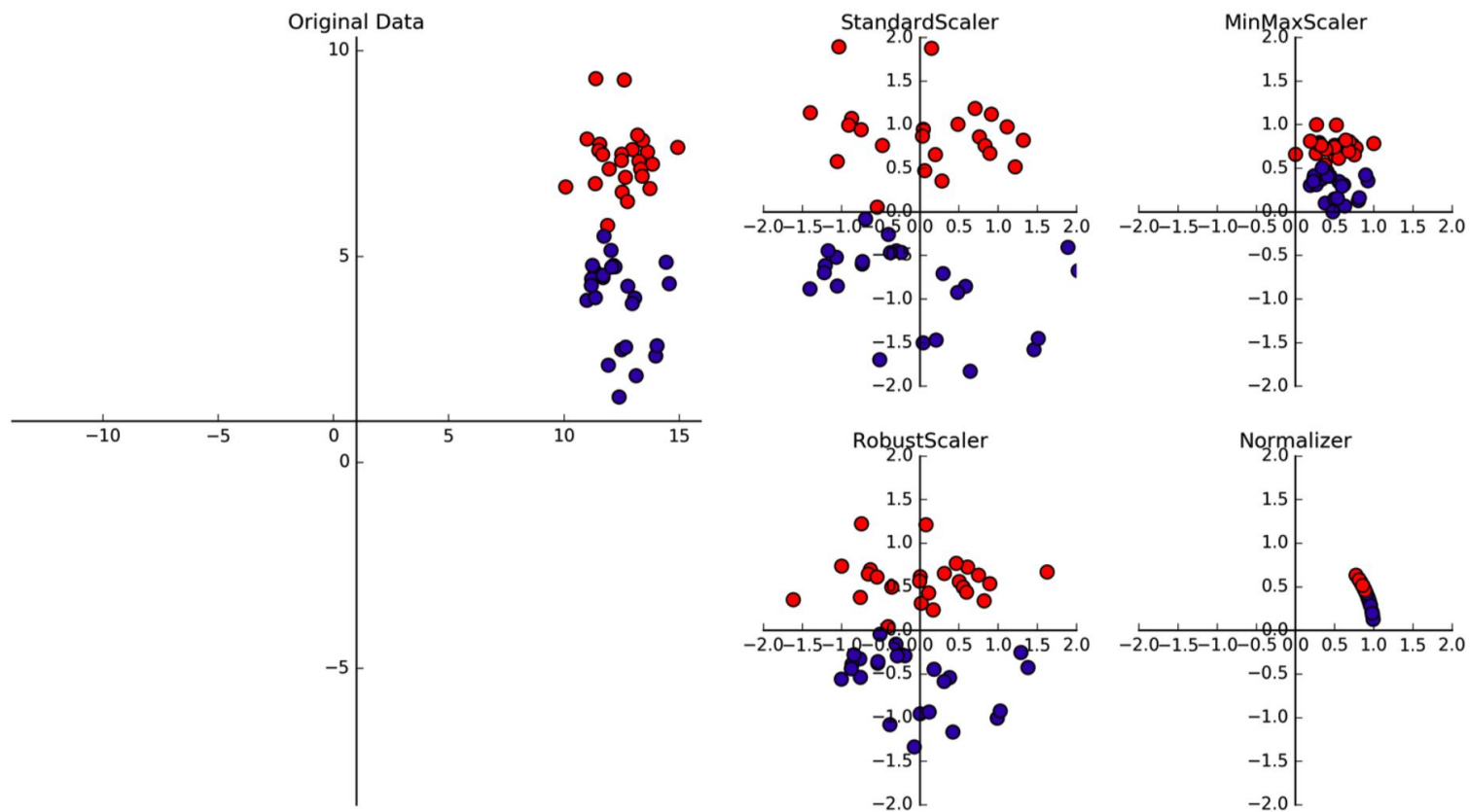
Feature Scaling:

e.g., *standardization*:
$$Z = \frac{X_{ik} - \mu_k}{\sigma_k} \quad (\text{use same parameters for the test/new data!})$$

- Faster convergence (gradient descent)
- Distances on same scale (k-NN with Euclidean distance)
- Mean centering for free
- Normal distributed data
- Numerical stability by avoiding small weights

Sampling:

- Random split into training and validation sets
- Typically 60/40, 70/30, 80/20
- Don't use validation set until the very end! (overfitting)



■ **Standardization: To transform data so that it has zero mean and unit variance. Also called scaling**

- Use function `sklearn.preprocessing.scale()`
- Parameters:
 - ***X***: Data to be scaled
 - ***with_mean***: Boolean. Whether to center the data (make zero mean)
 - ***with_std***: Boolean (whether to make unit standard deviation)

■ **Normalization: to transform data so that it is scaled to the [0,1] range.**

- Use function `sklearn.preprocessing.normalize()`
- Parameters:
 - ***X***: Data to be normalized
 - ***norm***: which norm to use: l1 or l2
 - ***axis***: whether to normalize by row or column
- and Normalization

```
>>> from sklearn import preprocessing
>>> import numpy as np
>>> X = np.array([[ 1., -1.,  2.],
...               [ 2.,  0.,  0.],
...               [ 0.,  1., -1.]])
>>> X_scaled = preprocessing.scale(X)

>>> X_scaled
array([[ 0.   ..., -1.22...,  1.33...],
       [ 1.22...,  0.   ..., -0.26...],
       [-1.22...,  1.22..., -1.06...]])
```

■ In scikit-learn, this is referred to as “Imputation”

■ Class be used `sklearn.preprocessing.Imputer`

○ Important parameters:

- ***strategy***. What to replace the missing value with: *mean / median / most_frequent*
- ***axis***: *Boolean. Whether to replace along rows or columns*

○ Attribute:

- ***statistics_*** : The imputer-filled values for each feature

○ Important methods

- ***fit(X[, y])*** Fit the model with X.
- ***transform(X)*** Replace all the missing values in X.

```
>>> import numpy as np
>>> from sklearn.preprocessing import Imputer
>>> imp = Imputer(missing_values='NaN', strategy='mean',
axis=0)
>>> imp.fit([[1, 2], [np.nan, 3], [7, 6]])
Imputer(axis=0, copy=True, missing_values='NaN', strategy=
'mean', verbose=0)
>>> X = [[np.nan, 2], [6, np.nan], [7, 6]]
>>> print(imp.transform(X))
[[ 4.         2.        ]
 [ 6.         3.666...]
 [ 7.         6.        ]]
```

- Using class **sklearn.utils.resample**
 - Important parameters:
 - ***n_sample***: No. of samples to keep
 - ***replace***: Boolean. Whether to resample with or without replacement
 - Returns sequence of resampled views of the collections. The original arrays are not impacted.
- Another useful class is `sklearn.utils.shuffle`

- Scikit-learn doesn't have a direct class that performs discretization.
- Can be performed with `cut` and `qcut` functions available in pandas.

- The `sklearn.feature_selection` module implements feature selection algorithms.
- Some classes in this module are:
 - ***GenericUnivariateSelect***. Univariate feature selector based on statistical tests.
 - ***SelectKBest***. Select features according to the k highest scores.
 - ***RFE***. Feature ranking with recursive feature elimination.
 - ***VarianceThreshold***. Feature selector that removes all low-variance features.
- Scikit-learn does not have a CFS implementation, but RFE works in somewhat similar fashion.

■ The `sklearn.decomposition` module includes matrix decomposition algorithms, including PCA

■ `sklearn.decomposition.PCA` class

○ Important parameters:

➤ ***n_components***: No. of components to keep

○ Important attributes:

➤ ***components_***: Components with maximum variance

➤ ***explained_variance_ratio_***: Percentage of variance explained by each of the selected components

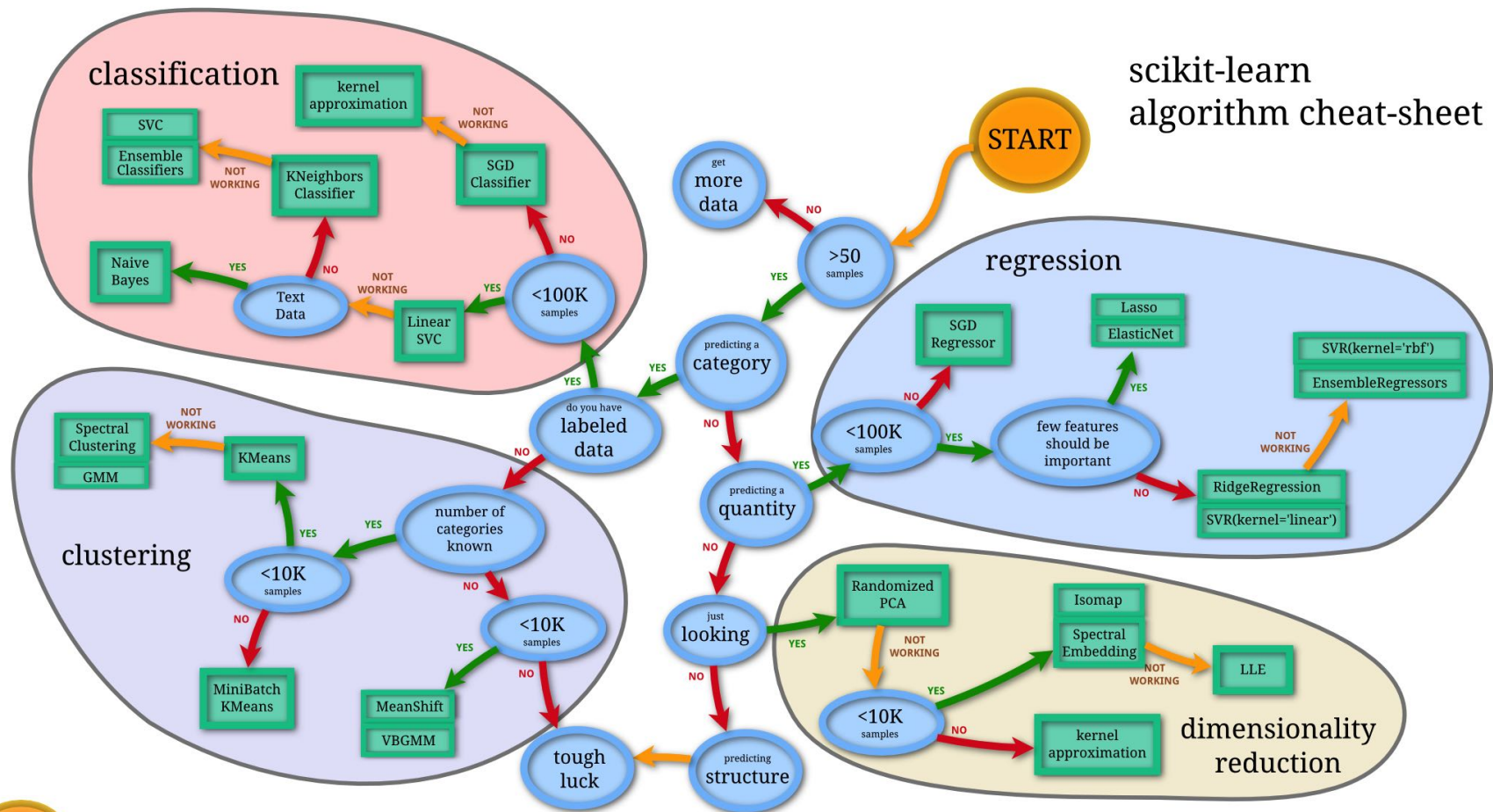
○ Important methods

➤ ***fit(X[, y])*** Fit the model with X.

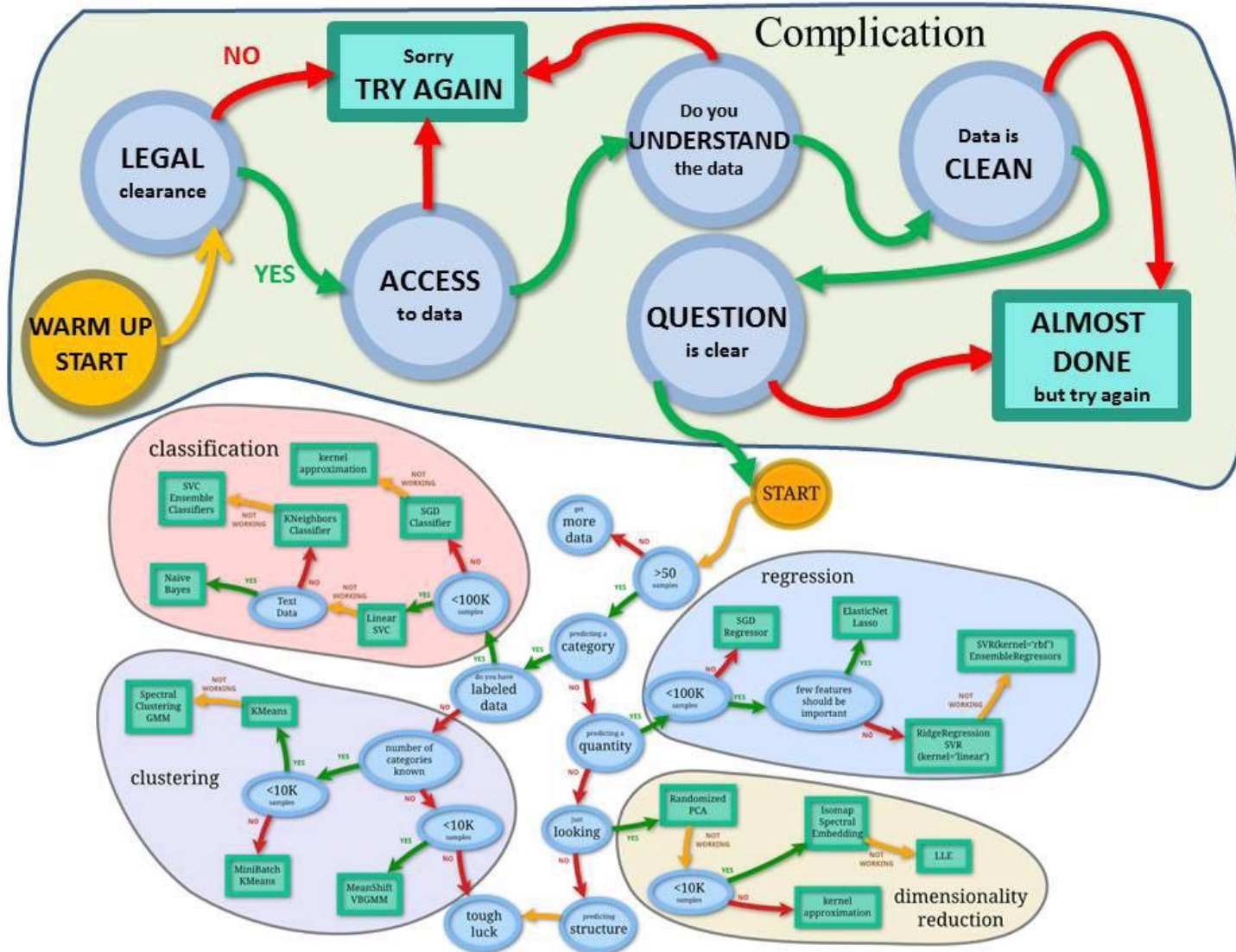
➤ ***score_samples(X)*** Return the log-likelihood of each sample

➤ ***transform(X)*** Apply the dimensionality reduction on X.

3. Algorithm

scikit-learn
algorithm cheat-sheet

Back



No Free Lunch

1. Our model is a simplification of reality
2. Simplification is based on assumptions (model bias)
3. Assumptions fail in certain situations

“No one model works best for all possible situations.”

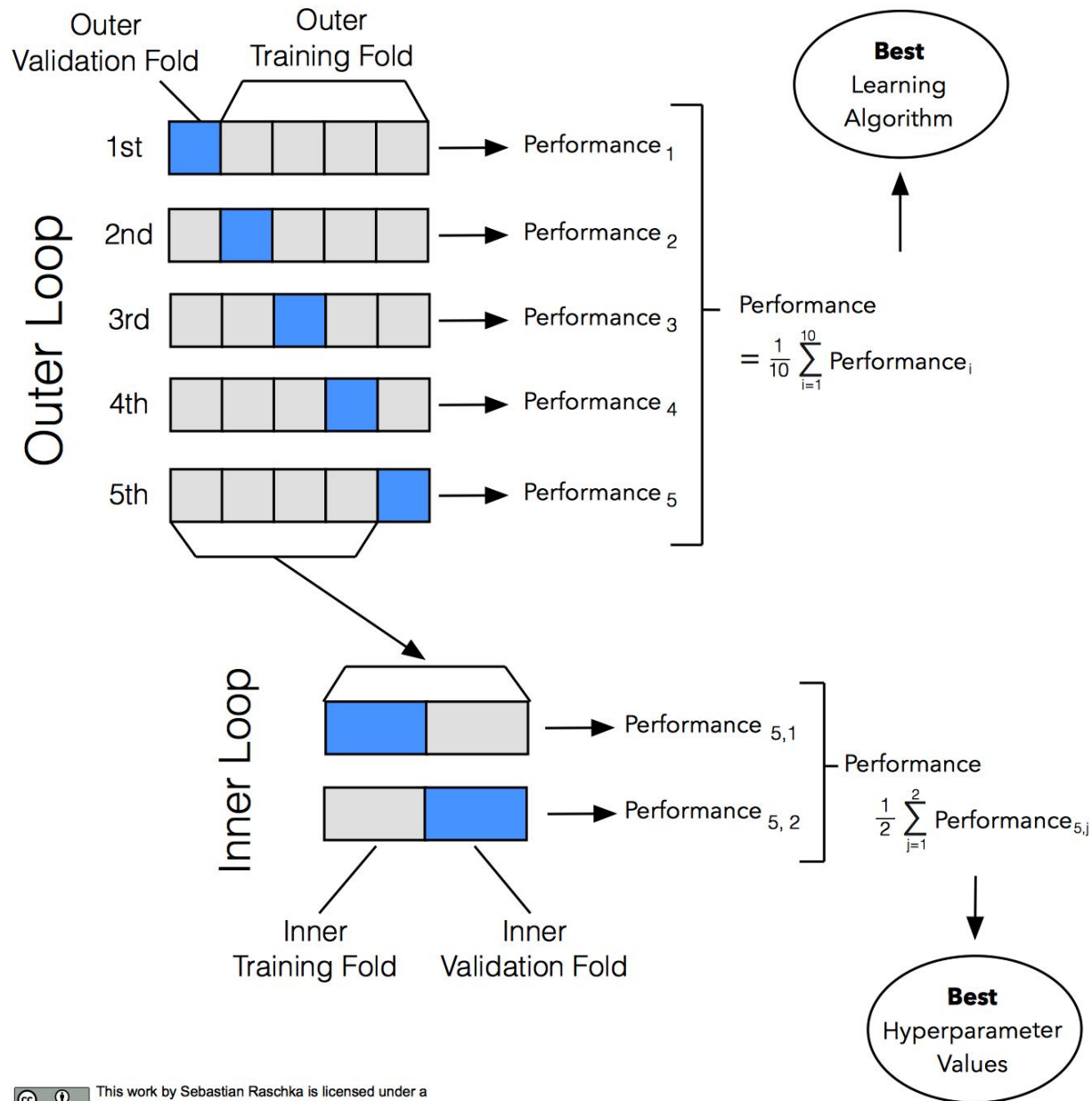
The **supervised** learning no-free-lunch theorems. (2002)

D. H. Wolpert.

Baseline Model

현실에는 Imbalanced Data가 아주 많음

Cross Validation



Holdout 방식과 비교

무작위로 데이터를 나누기 때문에 학습이 안되는 데이터가 있을 수 있음
우연히 평가가 좋게 혹은 나쁘게 나올 수 있음

➤ CV는 모든 데이터 활용

60~80%로 training, test data 분리

➤ 10fold 90% : 정확도가 높을 가능성, 속도는 fold만큼 느림

Training 데이터에 대한 모델 정확도를 예측

overfitting, underfitting 확인

Hyper parameter Tuning시 사용

CV로 얻은 모델은 최종 모델이 아님

Hyper Parameter Tuning

모델의 일반화 성능을 최대로 만드는 매개변수를 찾는 작업

- What is the size and dimensionality of training set?
- Is the data linearly separable?
- How much do I care about computational efficiency?
 - Model building vs. real-time prediction time
 - Eager vs. lazy learning / on-line vs. batch learning
 - prediction performance vs. speed
- Do I care about interpretability or should it "just work well?"

<http://blog.kaggle.com/category/winners-interviews/>

Linear models in practice

- Good with $P > n$: because simple!
(work well with sparse data)
- Good on **very** large : because fast!
- Very robust, always use as baseline!
- Relatively interpretable
- Try adding interaction or polynomial features

When to use tree-based models

- Model non-linear relationships
- Single tree: very interpretable (if small)
- Random forests very robust, good benchmark
- Gradient boosting often best performance with careful tuning
- Doesn't care about scaling, no need for feature engineering!

When to use neural networks

- Feed-forward vanilla: lots of data, to get the last couple %.
- Convolutional neural nets: Images, audio, video
- Recurrent neural nets: sequence prediction
- Word-embeddings: text classification

Usually need lots of tuning and preprocessing.

Search CV

Grid, Randomized

Training Data : Model 학습

Validation Data : Model, Hyper parameter 선택

➤ Grid Search, Randomized Search

Test Data : Model 평가

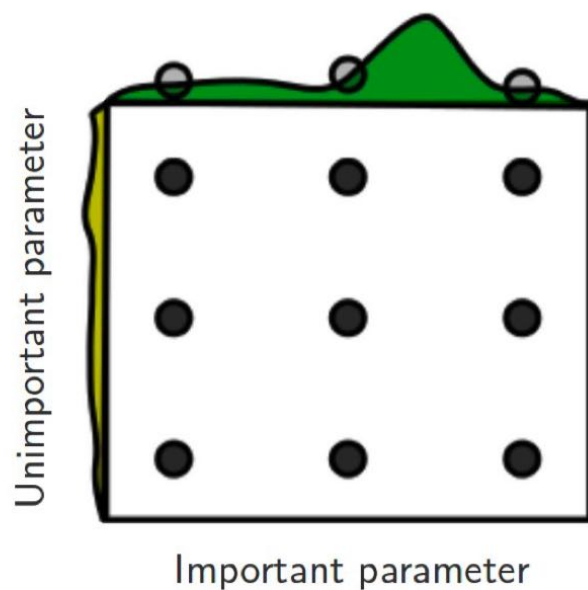
정확도가 높은 모델보다 목적에 따른 모델 선택

➤ 올바른 평가 지표 선택 필요

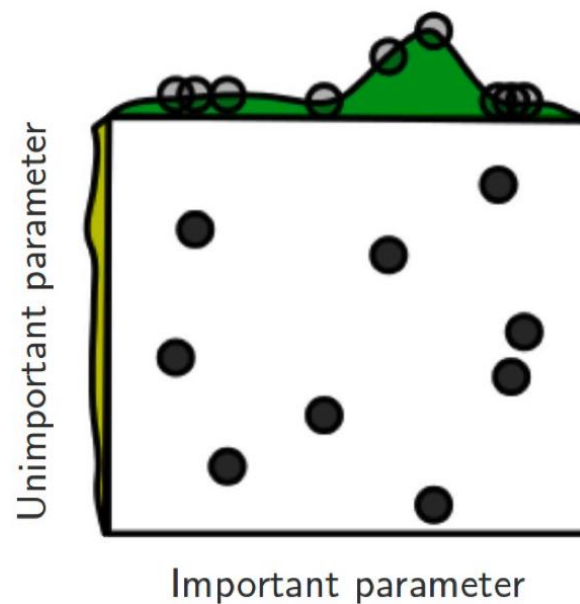
Gavin C. Cawley, Nicola L. C. Talbot,
**"On Over-fitting in Model Selection
and Subsequent Selection Bias in
Performance Evaluation",**
Journal of Machine Learning Research,
11(Jul):2079–2107, 2010.

Parameter Search

Grid Layout



Random Layout



Random Search for Hyper-Parameter Optimization (2012)

James Bergstra. Yoshua Bengio

max_features

n_estimators

	all	sqrt	log2	...
2	<code>max_features="all"</code> <code>n_estimators="2"</code>	<code>max_features="sqrt"</code> <code>n_estimators="2"</code>	<code>max_features="log2"</code> <code>n_estimators="2"</code>	...
5	<code>max_features="all"</code> <code>n_estimators="5"</code>	<code>max_features="sqrt"</code> <code>n_estimators="5"</code>	<code>max_features="log2"</code> <code>n_estimators="5"</code>	...
9	<code>max_features="all"</code> <code>n_estimators="9"</code>	<code>max_features="sqrt"</code> <code>n_estimators="9"</code>	<code>max_features="log2"</code> <code>n_estimators="9"</code>	...
⋮	⋮	⋮	⋮	⋮

a

Pick parameter combinations

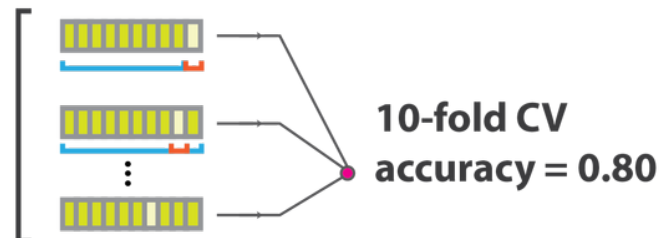
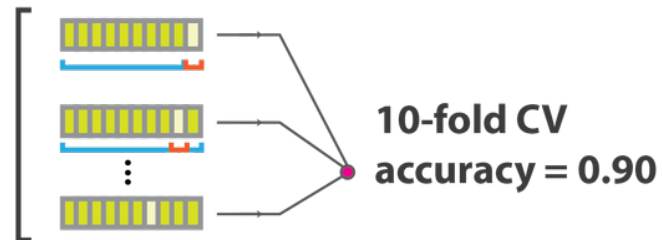
parameter combination that defines **model 1**

parameter combination that defines **model 2**

parameter combination that defines **model n**

b

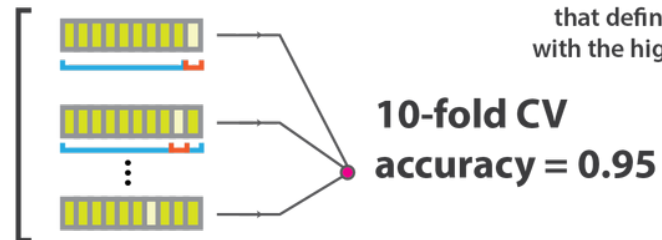
Perform k-fold CV



⋮

c

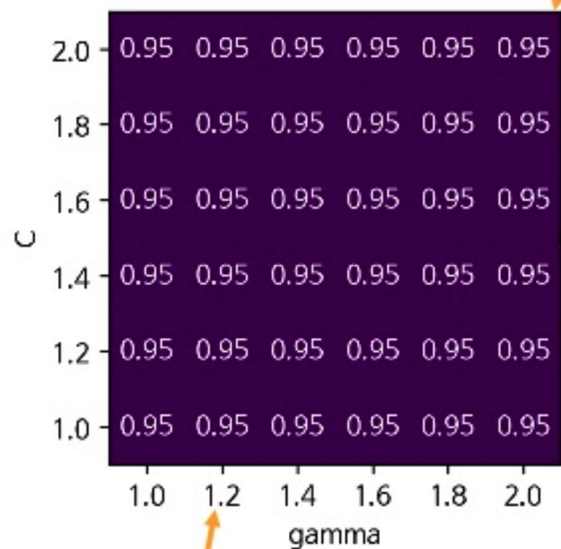
Repeat.



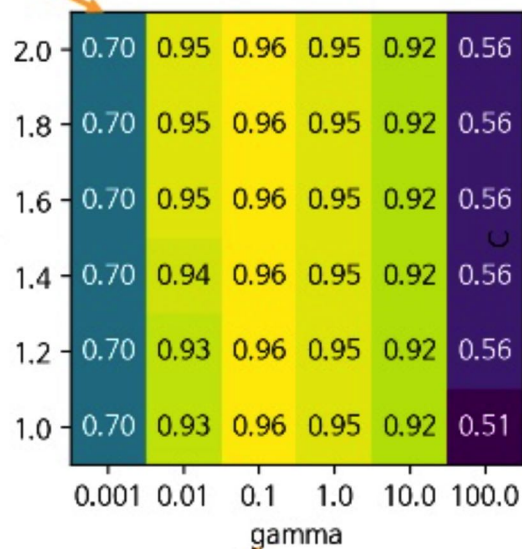
d

Pick the set of parameters that define the model with the highest accuracy

변화 없음
(절적하지 않은 범위이거나
중요하지 않은 매개변수)



더 극단적인 값을 적용할 필요 있음



gamma 매개변수만 영향을 미침

검색 영역 조정 필요

